MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

James Gabriel P. Mabagos
Bachelor of Science in Computer Science

Mark Lawrence M. Quibot
Bachelor of Science in Computer Science

Senior Thesis submitted to the faculty of the
Department of Computer Science
College of Computer Studies, Ateneo de Naga University
in partial fulfillment of the requirements for their respective
Bachelor of Science degrees

Project Advisor: Marianne A. Tolentino
Estrella H. Montealegre
Michelle Elija B. Santos
Ian Peter L. Lastimosa

September 5 2025
Naga City, Philippines

Keywords: Imperfect Information, DQN, Multi-Agent

The Senior Project entitled

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

developed by

James Gabriel P. Mabagos

Bachelor of Science in Computer Science

Mark Lawrence M. Quibot

Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees has been rigorously examined and recommended for approval and acceptance.

Estrella H. Montealegre
Panel Member
Date signed: _____

Michelle Elija B. Santos
Panel Member
Date signed: _____

Ian Peter L. Lastimosa
Panel Member
Date signed: _____

Marianne A. Tolentino
Project Advisor
Date signed: _____

The Senior Project entitled

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

developed by

James Gabriel P. Mabagos
Bachelor of Science in Computer Science

Mark Lawrence M. Quibot
Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees is hereby approved and accepted by the Department of Computer Science, College of Computer Studies, Ateneo de Naga University.

Lowie Vincent S. Bisana MIT
Chair, Department of Computer Science
Date signed: _____

Joshua C. Martinez MIT
Dean, College of Computer Studies
Date signed: _____

Declaration of Original Work

We declare that the Senior Project entitled

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

which we submitted to the faculty of the

Department of Computer Science, Ateneo de Naga University

is our own work. To the best of our knowledge, it does not contain materials published or written by another person, except where due citation and acknowledgement is made in our senior project documentation. The contributions of other people whom we worked with to complete this senior project are explicitly cited and acknowledged in our senior project documentation.

We also declare that the intellectual content of this senior project is the product of our own work. We conceptualized, designed, encoded, and debugged the source code of the core programs in our senior project. The source code of third party APIs and library functions used in our program are explicitly cited and acknowledged in our senior project documentation. Also duly acknowledged are the assistance of others in minor details of editing and reproduction of the documentation.

In our honor, we declare that we did not pass off as our own the work done by another person. We are the only persons who encoded the source code of our software. We understand that we may get a failing mark if the source code of our program is in fact the work of another person.

James Gabriel P. Mabagos
4 - Bachelor of Science in Computer Science

Mark Lawrence M. Quibot
4 - Bachelor of Science in Computer Science

This declaration is witnessed by:

Marianne A. Tolentino
Project Advisor

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

by

James Gabriel P. Mabagos and Mark Lawrence M. Quibot

Project Advisor: Marianne A. Tolentino

Department of Computer Science

ABSTRACT

This research presents MARQ, a Multi-Agent Rainbow Deep Q-Networks, designed to master imperfect information games through the board game Stratego. MARQ leverages a Deep Recurrent Q-Network (DRQN) architecture and league-based self-play training to address the large state space of Stratego and hidden information challenges. The architecture integrates an Attention as Recurrent Neural Network (AAREN) module that produces learned embeddings from observed opponent behavior, enabling implicit piece identity inference through end-to-end training. The system employs Distributional Reinforcement Learning with Noisy Networks to balance strategic unpredictability and stability. The methodology involves developing a game environment and training the AI agent through multi-channel state representations, incorporating AAREN-generated embeddings for hidden information modeling and weighted piece values computed through systematic power analysis to optimize decision making under uncertainty. A structured participant study introduces enthusiasts without prior Stratego experience to create a synchronized learning environment, combining quantitative metrics including win rate percentage, average game length, and decision-making speed, with qualitative usability assessments across performance, reliability, and applicability dimensions. Through this study MARQ aims to deliver strategic decisions comparable to human opponents.

ACKNOWLEDGEMENTS

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

Chapter 1

Introduction

1.1 Project Context

Games have a history of being a metric for how Artificial Intelligence (AI) progresses and performs in the current time [38]. Many real-world problems, from military tactics to multiplayer games, require intelligent decision-making in environments where all information is not available. A common example is a feature found in many multiplayer games known as the "fog of war" [51], where the game environment is not fully revealed, which makes it an example of an imperfect information game [41]. Imperfect information games create a scenario where players develop unconventional strategies without having full knowledge of the game's environmental state [51]. Solving these kinds of problems and determining the optimal strategy remains a significant challenge for Artificial Intelligence [27].

This study focuses on mastering imperfect information games through Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN). In MARL, multiple agents learn and adapt in the same environment, making each decision based on its own observations and rewards [29]. With DQN, the agents can utilize deep neural networks to approximate the Q-function that maps state-action pairs for the expected reward. This study introduces the MARQ (Multi-Agent Rainbow Deep Q-Networks) framework, which extends the basic DQN architecture with Rainbow enhancements including distributional value estimation and noisy exploration networks [20]. The framework integrates an AAREN (Attention as Recurrent Neural Network) module [18] for processing extended observation sequences and producing learned embeddings that enable implicit inference of hidden op-

ponent pieces through end-to-end training. League-based self-play training [35] ensures that agents develop robust strategies by competing against diverse historical opponents. This approach is suited well for Stratego, where agents are trained to compete against each other, developing strategies through repeated experience while reasoning under uncertainty about the opponent's setup and the environment's hidden state.

1.2 Purpose and Description

This research aims to evaluate whether Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN) can be used to develop intelligent agents capable of mastering imperfect information games. By focusing on the limited information available to players in environments such as the board game Stratego, this study seeks to address the challenge of making decisions under uncertainty while incorporating strategic planning. Although existing tools for Stratego provide some analytical support, they are often limited in fostering deeper reasoning about the game's tactical complexities.

To advance beyond these limitations, this research seeks to provide an algorithmic foundation for future analytical systems while also emphasizing the development of critical thinking in solving imperfect information games. By modeling adaptive strategies and reasoning under uncertainty, the study highlights how artificial intelligence can mirror and enhance human-like logical reasoning, offering valuable insights for both game analysis and broader decision-making domains.

1.2.1 Research Questions

This study is guided by three main research questions:

1. How does the MARQ framework perform in terms of convergence speed and win-rate against heuristic baselines in a partially observable environment?

2. Does the MARQ framework demonstrate statistically significant superiority over standard DQN and rule-based agents across varying opponent difficulty tiers?

3. What is the impact of integrating the AAREN module on the agent's ability to infer hidden opponent piece configurations compared to approaches without explicit belief state modeling?

1.3 Objectives

The main objective of this study is to develop and evaluate Multi-Agent Reinforcement Learning with Deep Q-Networks for learning appropriate strategies in an imperfect information game, where the agents operate under hidden state variables to improve decision-making, adaptability, and strategic reasoning against both human and AI opponents. This can be achieved through the following:

- Finding ideal algorithms to be implemented in perfecting Stratego.
- Design and implement a game environment for Stratego
- Build and implement a training model using Multi-Agent Reinforcement Learning with Deep Q-Networks and AAREN.
- Train and optimize the AI agent to respond to varying game states, hidden information, and opposing strategies.

1.4 Scope and Limitation

The study is limited to the development of an AI agent for imperfect information games, Stratego being the test environment. The scope includes the design and implementation of Stratego as a game environment, the development of a training framework using Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN) using AAREN, and the training and optimization of the AI agent to adapt to differences in game states. The evaluation of the agent's performance would primarily be on Stratego's environment. The trainings are limited to the personal devices of the researchers and would not rely on cloud-based computing resources. Since the AI is going to learn alongside the community the player and AI head-to-head will be analyzed mostly qualitative. The goal will be achieving the same human critical thinking in the model.

1.5 Significance of the Study

The development of the MARQ framework addresses limitations in current imperfect information game AI, where existing systems either require large computational resources or employ model-free approaches that limit strategic depth. The MARQ framework's significance lies in its implicit opponent modeling through AAREN-generated embeddings, and hybrid DQN architecture that balances

strategic depth with computational tractability. The synchronized learning evaluation framework provides insights into AI adaptability against improving human opponents, with implications extending beyond gaming to domains characterized by incomplete information and strategic interaction, including cybersecurity, financial trading, and strategic planning.

1.6 Definition of Terms

AAREN (Attention as Recurrent Neural Network)
A neural architecture that combines the parallel training efficiency of Transformers with the sequential processing of RNNs. In the MARQ framework, AAREN processes observation sequences to produce learned embeddings that encode implicit piece identity inference.

Counterfactual Regret Minimization (CFR)
An iterative algorithm that approximates Nash equilibrium by minimizing regret, widely applied in imperfect information games such as Poker and Stratego.

Deep Q-Networks (DQN)
A reinforcement learning algorithm that combines Q-learning with deep neural networks, enabling agents to approximate optimal decision-making in complex environments.

Distributional Reinforcement Learning
An approach that models the full distribution of returns rather than just the expected value. The MARQ framework uses C51 (Categorical DQN) with 51 atoms to capture uncertainty in value estimates.

Fog of War
A game mechanic used in strategy games that conceals parts of the map or opponent actions, requiring players to utilize prediction and information-gathering strategies to reduce uncertainty.

Imperfect Information Games

Games where players have incomplete knowledge of the current game state, requiring them to make strategic decisions under uncertainty (e.g., Stratego, Poker, Multiplayer Online Battle Arenas).

Information Set Monte Carlo Tree Search (IS-MCTS)

A search algorithm for imperfect information games that samples consistent game states from belief distributions and performs tree search on information sets rather than individual states.

League Training

A self-play training paradigm where agents compete against a diverse pool of historical opponents, preventing strategy cycling and ensuring robust generalization across different playing styles.

Multi-Agent Reinforcement Learning (MARL)

An extension of reinforcement learning in which multiple agents interact within the same environment, learning coordinated or competitive strategies through simultaneous training.

Nash Equilibrium

A game-theoretic concept where no player can gain an advantage by unilaterally changing their strategy, often used as a benchmark for decision-making in imperfect information games.

Perfect Information Games

Games in which all players have complete knowledge of the current game state and available moves, such as Chess and Go.

AAREN Embeddings

Embedding vectors produced by the AAREN module from observed opponent action sequences.

Rainbow DQN

An enhanced DQN architecture that combines multiple improvements including distributional RL, noisy networks for exploration, and dueling network architecture for improved stability and sample

efficiency.

Recurrent Neural Network (RNN)

A type of neural network designed to process sequential data by maintaining memory of previous inputs, useful in imperfect information games for recalling past moves or revealed information.

Reinforcement Learning (RL)

A machine learning paradigm where agents learn optimal strategies through trial-and-error interactions with the environment, guided by rewards and penalties.

Stratego

A two-player strategy board game characterized by hidden information and asymmetric piece abilities, where the objective is to capture the opponent's flag or render them immobile.

Chapter 2

Review of Related Literature

2.1 Artificial Intelligence in Imperfect Information Games

Games can be classified as an imperfect information game by having incomplete information about the current game state for both players [38, 29]. An example would be in MOBAs (Multiplayer Online Battle Arena), the fog of war mechanic hides portions of the map, including objectives and enemy positions, from both teams. By placing a tool, such as a ward in League of Legends or an observer in Dota 2, the player gains a temporary vision in the map for strategic planning [13]. This feature engages players to create strategic reasoning under certainty, utilizing prediction, coordination, and resource management to gain informational advantages over opponents. Games have a history of being a metric on how AI progresses and performance in the current time [38, 21]. The case is the same for both perfect information and imperfect information games [38]. For perfect information games, like chess, they primarily use Alpha-Beta pruning, a search algorithm that uses trees to explore possible chess moves and prunes the branches that will not affect the final decision [31]. In imperfect information games, a search algorithm like Alpha-Beta Pruning would be difficult to implement because the algorithm is made for perfect information games [37]. A popular algorithm that is used in imperfect information games would be Monte Carlo Tree Search with Reinforcement Learning because MCTS has integration with neural network systems like AlphaZero [33, 8]. In terms of reinforcement learning, the training focuses on trial and error to provide the best possible decision [54]. Continuing development of AI algorithms in imperfect information games advances

our understanding of decision-making under uncertainty and promotes strategic planning.

2.1.1 Student of Games

A unified learning algorithm that can support both perfect information and imperfect information is Student of Games. Combining self-play learning, strategic search, and principles from game theory [38]. SoG achieved strong performance in Chess and Go in perfect information games and in imperfect information games like heads-up no-limit Texas hold 'em poker, and defeated the state-of-the-art agent in Scotland Yard. When comparing to MARL, SoG supports and has been tested in both game types and uses game theoretic reasoning, which computes or approximates the Nash equilibria. MARL with DQN uses trial and error learning and self-play learning, which relies on the exploration of the environment. MARL has also been tested in complex video games [29], unlike SoG, which has not been tested in video games.

2.1.2 Deep Q-Networks and Dueling Architectures

Deep Q-networks (DQN) and their variants have become a standard framework for decision-making in complex environments with high-dimensional observations. Hessel et al. (2021) combined several independent improvements to DQN—such as double Q-learning, prioritized replay, dueling networks, and noisy nets—into a unified agent called Rainbow, demonstrating state-of-the-art performance on competitive benchmarks [20]. This demonstrates that carefully integrating architectural and algorithmic improvements can substantially improve both data efficiency and final performance in value-based deep RL.

A key architectural component of Rainbow is the dueling network architecture, proposed by Wang et al. (2016) and detailed in modern surveys [20, 41]. This design splits the Q-function into a state-value stream and an advantage stream. This split allows the network to generalize across actions and improves policy evaluation, particularly in environments with many similar-valued actions. Our MARQ model builds on this idea by feeding the shared ResNet-based representation into dueling value and advantage streams, which is particularly beneficial in board games where many positions have similar strategic value.

2.1.3 Multi-Agent Reinforcement Learning

Reinforcement Learning can solve complex problems through trial and error, by learning from the environment to be able to provide optimal decisions [29]. This can be the case for Single-Agent Reinforcement Learning(SARL). SARL has its limits in solving many real-world problems. Multi-Agent Reinforcement Learning(MARL) was introduced as a solution for the challenges of SARL, by having multiple agents in an environment that interact with each other, the decisions they could make would be optimized [29]. Optimized, meaning that these agents go through trial and error together to make the best decision given the reward. As for the implementation of this in Stratego, two agents would compete with each other to know the optimal decisions given the environment, which has imperfect information and opposing strategies. As MARL has its advantages, it also has challenges. The first would be Non-Stationarity, and the second is scalability, as for non-stationarity, each agent learns simultaneously, meaning the environment constantly changes [11]. For scalability, as more agents are being used, the computational power rises, making it expensive [28, 11]. Recent work on Ataraxos demonstrates that these challenges can be addressed through careful regularization: by constraining policy updates toward both exploratory (uniform) and stable (target network) anchors with time-varying coefficients, agents can achieve superhuman performance in complex imperfect-information games while maintaining training stability [39].

2.1.4 Counterfactual Regret Minimization

Counterfactual Regret Minimization(CFR) is an iterative algorithm that approximates the Nash equilibria commonly used on imperfect information games like poker and Stratego [28, 48]. The Nash equilibria are what CFR approximates to make the best possible decision with minimal regret. Regret in this scenario is the what-ifs of the algorithm if the algorithm chooses the other decision. For CFR to make decisions, it repeatedly minimizes regret to approximate the Nash equilibria [28]. Other works have made variations of the base CFR, showing improved convergence rate, but require a significant amount of effort by researchers. A framework is proposed named AutoCFR, instead of relying on manually crafted algorithms, it automatically designs new CFR variants using an outer loop to search over algorithm designs and an inner loop to test them on equilibrium finding tasks [47].

2.1.5 Opponent Modeling in Imperfect-Information Games

In imperfect-information games, an agent's optimal strategy depends not only on the environment but also on the behavior of the opponent. Classical opponent modeling often builds models from observed action frequencies, and computes best responses in a game-theoretic framework. Li et al. (2024) formalize opponent-model search in games with incomplete information, providing algorithms to compute optimal or robust strategies given various forms of opponent knowledge [27].

In the deep RL setting, modern approaches encode observations of opponents directly into the network trunk, jointly learning a policy and an opponent model without explicitly predicting actions [41]. This shows that learning implicit opponent representations from interaction traces can be more effective than handcrafted models, especially in high-dimensional state spaces. Our AAREN module follows this tradition of implicit opponent modeling but is specialized to board games with hidden piece identities. Instead of maintaining explicit probability distributions over opponent piece types, AAREN aggregates observed opponent action sequences into dense embedding vectors. These vectors are then concatenated as additional input channels to the Q-network, allowing the agent to reason about the opponent's likely pieces and strategy in a learned representation space.

2.2 RNN

Recurrent Neural Network is a type of Neural Network designed to process sequential data. Unlike common Neural Networks, RNNs have a memory because the information passed to the next step is based on the memory from the previous step. RNN can be applied to imperfect information games such as Stratego because of its memory. By using the memory to remember the pieces that have been revealed, the decision could be optimized [33].

2.2.1 History Aggregation and State Representation Learning

Learning from histories is a central challenge in partially observable and non-Markovian environments. Wang et al. (2026) show how to systematically construct non-Markovian decision processes by aggregating interaction histories into state representations, providing an effective way to add temporal dependencies into RL [43]. By explicitly modeling history via learned aggregators, an agent can better handle complex temporal structures.

More broadly, representation learning for RL has explored the use of embeddings to generalize across states and goals. Echchahed and Castro (2025) highlight in their survey that separating dynamics from rewards allows policies to be transferred across tasks through learned representations [16]. Our AAREN embeddings can be viewed as a form of history-dependent state representation that encodes the opponent's action history into a compact vector to augment the board state. This approach is tailored to the specific structure of imperfect-information board games where opponent behavior reveals hidden state. By learning these embeddings jointly with the Q-network, we avoid the need to handcraft belief distributions and instead let the network infer implicit piece identities from observed behavior.

2.2.2 LSTM

Long Short-Term Memory is a popular RNN variant that has the capabilities of RNN but also processes data with long-term dependencies [2]. LSTM mitigates the vanishing gradient problem that RNN faces. The vanishing gradient problem is basically as in the name, vanishing gradient, as color in a gradient, the message in this case starts strong and gradually fades away when it passes through each layer, making the learning weak or close to zero. LSTM can be used to solve imperfect information games because of its capability to process data with long-term dependencies while having the memory of RNN.

2.2.3 Aaren

Aaren, known as Attention as a recurrent neural network, combines the parallel training efficiency of Transformers with the streaming update efficiency of RNNs [18]. Aaren, like transformers, can be trained in parallel as well as RNN, can process sequential data with memory. In the study, Aaren was tested in event forecasting, time series forecasting, and classifications. Aaren achieved similar accuracy to Transformers but was more time and memory-efficient.

2.2.4 Residual Networks and Self-Attention in Board Games

Residual networks (ResNets) have become a common backbone in modern board-game RL systems. AlphaZero-style agents for Go, chess, and Stratego use a deep tower of residual convolutional blocks as the network trunk, enabling stable training of deep networks that can capture spatial dependencies

on the board [8]. Modern architectures often mix these residual blocks with self-attention layers to bridge local and global knowledge. This hybrid architecture significantly improves playing strength in board games by better handling long-range patterns [4].

Because of these results, our architecture adopts a ResNet backbone with spatial self-attention. This allow each board position to attend to all other positions, capturing long-range piece relationships that are difficult to represent with purely convolutional architectures. We extend this line of work by combining self-attention with a dueling Q-network and a learned opponent embedding, addressing the additional challenges of imperfect information and opponent modeling in Stratego.

2.3 Stratego

The origin of Stratego can be traced to a traditional board game called Jungle, where instead of human pieces instead it used animals. In the jungle, the pieces are not hidden, making it a perfect information game. Stratego is a two-player board game where players each have a 40-piece army. In the tabletop version of Stratego, one player is assigned to the blue army and the other player is assigned to the red army. Each army has a flag assigned to it, which cannot be moved when the game starts. There are a total of 80 troop pieces, while the total number of variations is 12. The pieces rank range from 1 to 10, and a piece that's not numbered is a bomb and the flag. The player also has time to move pieces before the game starts to make use of the fog of war and create creativity and optimal positions on the board. As the pieces have ranks, the higher the number, the higher the power. There is an instance where the lower-ranked can defeat a higher-ranked ranked, which is the piece that has a rank of one, named the spy, can defeat the strongest piece that has a rank of ten, named the marshal, if the spy attacks first. As for the bombs, almost all pieces are vulnerable except the miner, the only one who can defuse the bomb. And the pieces that cannot be moved are the bomb and the flag. Another piece that has a special ability is the scout; it can move either horizontally or vertically, not just one square. If both pieces have the same number, and the piece attacks, both of them would be eliminated. There are multiple win conditions in Stratego. The obvious one would be capturing the flag, another one would be that the opponents have no mobile pieces, another would be that there are no miners that can defuse the bomb to capture the flag, and lastly, the opposing player had the time run out.

2.3.1 Deep Reinforcement Learning in Stratego

Stratego is a popular imperfect-information board game where players must reason about hidden piece identities over long time horizons. Pérolat et al. (2022) demonstrated with DeepNash that a model-free multiagent RL approach can reach human expert level in Stratego, using deep convolutional networks trained with game-theoretic regularization [35]. Their architecture relies heavily on ResNet-style networks to process board features under imperfect information. More recent work has further improved these baselines by combining self-play reinforcement learning with test-time search, achieving superhuman performance [39].

Our MARQ framework is closely related to these systems in its use of a deep ResNet backbone, but we explicitly incorporate opponent modeling via AAREN embeddings. By basing the agent on the opponent's observed actions, we allow the agent to adapt to specific opponents and exploit behavioral patterns. This adds to the equilibrium-style play of prior models with learned opponent representations.

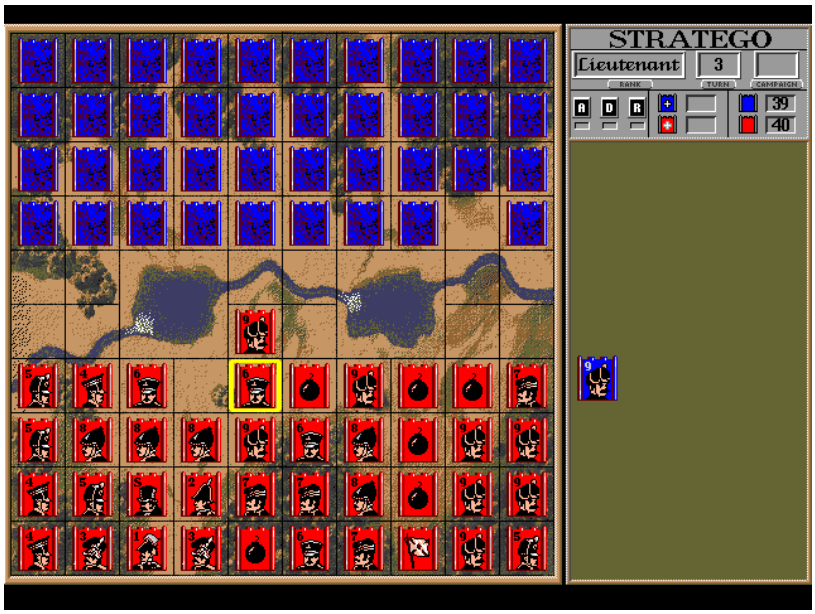


Figure 2.1: Stratego Game board.

2.4 Board Games in Community Building

Modern board games are becoming more widespread, expanding their market share each year. During COVID-19 lockdowns, board game sales increased because people were stuck at home with family and had free time. Also, playing board games during that time helped with stress, isolation, and anxiety. Certain games like Pandemic, a cooperative strategy board game where players work together to stop global disease outbreaks by discovering cures before time runs out, became more popular due to their reflection of the real-world scenario. Board game communities differ between the West and the East [19]. From the east, board games are considered social activities, while in the west, board games grew as hobbies, communities focusing on the gameplay and mechanics. In Hong Kong, there are two distinct board game communities named BGHK and Oasis. BGHK consists of Cantonese-speaking individuals who treat board games as social bonding. They even event sessions for "party game days" and "welcome newbies," emphasizing making friends rather than the game itself. While the Oasis group focuses on the hobby, playing newly released and complex games. Attracting players with the mechanics and novelty rather than just socializing. A study has been made to use board games as an answer to the problem of difficulty in creating a sense of "togetherness or kinship," and board games have advantages for improving the cognitive function of older people [3]. In the study, the Tresna Werdha Budi Pertiwi Social Home has a problem of having a sense of family among residents. By playing together, the elderly had consistent interaction with each other, reducing the feeling of loneliness and detachment. Board games not only provide entertainment but also memory stimulation and improve social bonds. Board games provide a face-to-face interaction requiring people to sit together at a table, unlike in video games, where players are often separate from each other [34]. Every shared experience creates a unique microenvironment where players enter a separate reality governed by the game's rules. In the space, the players share emotions and strategy. Sharing these emotions helps strengthen friendships, family bonds, and trust. Many board game players first connect through gaming with friends and family, which can branch out to dedicated groups or conventions, and expand their social circle.

2.5 Other Applications

An extension of MARL with DQN was demonstrated in robotics. In these environments, robots are trained to make decisions based on rewards. The study assigned different energy costs to each

robotic arm, allowing the robot to learn how to adjust its movement based on cost, using less energy when the reward is small and exerting more effort if the reward is appropriate [30]. MARL with Deep Q-learning was also used in traffic signal control. Applied a cooperative multi-agent DQN framework to manage six interconnected intersections, where each intersection acted as an independent learning agent. Using the SUMO simulator, each agent selected signal phases to minimize congestion and improve overall traffic flow. Their approach introduced a reward function based on the standard deviation of stopping vehicles across lanes, showing balanced lane utilization rather than simply reducing queue lengths. This design showed that MARL with DQN can enhance not only traffic efficiency but also environmental sustainability by reducing emissions and fuel consumption. In businesses in developing countries, it is essential that growth should be the top priority; however, this creates a problem in increasing taxation for improving infrastructure to further support the current population. Using MARL, we can create a model system where multi-agent RL acts as different parties that discover the optimal tax policies without needing to rely on traditional economic models [53].

2.6 Application to Other Game Environments

Although this study focuses on Stratego as the game environment, the MARQ framework can also be applied to other strategic and simulation-based games that involve decision-making and hidden information. Sports management titles such as Madden NFL's Dynasty Mode, NBA 2K's franchise mode, Esports Manager, and Football Manager feature complex systems of player development, transfer, and long-term planning, where agents must act without complete information about the future player performance and market behavior. In these environments, the MARQ framework could model AI agents capable of learning optimal trade strategies or budget allocations through reinforcement learning. The framework's belief-state modeling would allow the AI to check hidden player attributes or opponent strategies, while the Deep Q-Network could optimize sequential decisions across multiple simulated seasons.

Similarly, the framework could be applied to a game called Clash Royale, where players must make fast tactical decisions without full knowledge of their opponent's hand or resource level. The Probabilistic Belief State in this setting could estimate the likelihood of specific opponent cards being available, while the Deep Q-Network would select optimal actions for troop deployment and timing.

IS-MCTS could be used to search over possible opponent hands, enabling informed decision-making for lane control or counter-attack scenarios.

Chapter 3

Theoretical Frameworks

3.1 Stratego Game Domain

Stratego is an imperfect information board game with European roots, popularized in the Netherlands in the 1940s and released in North America in 1961 [35]. The game is played on a 10x10 board that includes two 2x2 "lake" areas in the center, which are impassable. Each player commands an army of 40 pieces of varying ranks, and during the game, the identity and rank of the pieces are kept secret from the opposing player. The objective is to capture the opponent's flag or eliminate all of their movable pieces.

Each army consists of 12 different types of pieces, with ranks ranging from 1 (the Spy) to 10 (the Marshal), alongside Bombs and a Flag. Higher-ranked pieces defeat lower-ranked ones during combat, except in special situations: the Spy can defeat a Marshal if attacking first, the Miner is the only piece that can defuse Bombs, and Scouts can move multiple squares in a straight line. There are 3 ways to win the game capturing the flag, immobilizing all movable enemy pieces, or forcing the opponent to run out of time.

The high theoretical state space of Stratego shows how difficult the game is to solve:

$$|\mathcal{S}| = \binom{40}{1, 1, 2, 3, 4, 4, 4, 5, 8} \times 10^{10} \times 2^{40} \approx 10^{535} \tag{3.1}$$

Perolat et al. [35] established the permutation for game state. The agent must use reasoning with limited information rather than trying to look at every possible outcome, making the computation

Figure 3.1: Stratego pieces.

intractable.

3.2 MARQ Framework Overview

The MARQ framework is a Multi-Agent Rainbow Deep Q-Networks system that trains agents through self-play and deep reinforcement learning [8]. Agents learn by competing against many different opponents, including their own previous versions and exploratory variants, within a league-based self-play system.

The central problem is reasoning about hidden information without enumerating every possible state. The number of possible game states exceeds 10^{535}, and initial setups reach 10^{66} per player [35]. MARQ handles this scale through implicit belief representations, probabilistic reasoning, and pattern recognition. Deep reinforcement learning estimates values. Attention mechanisms process game history. Mixed strategies prevent exploitation. The system keeps all of these computations fast enough to run in real time.

The architecture connects these components in a pipeline. The AAREN module takes raw game positions and move sequences, then produces learned embeddings that encode opponent piece

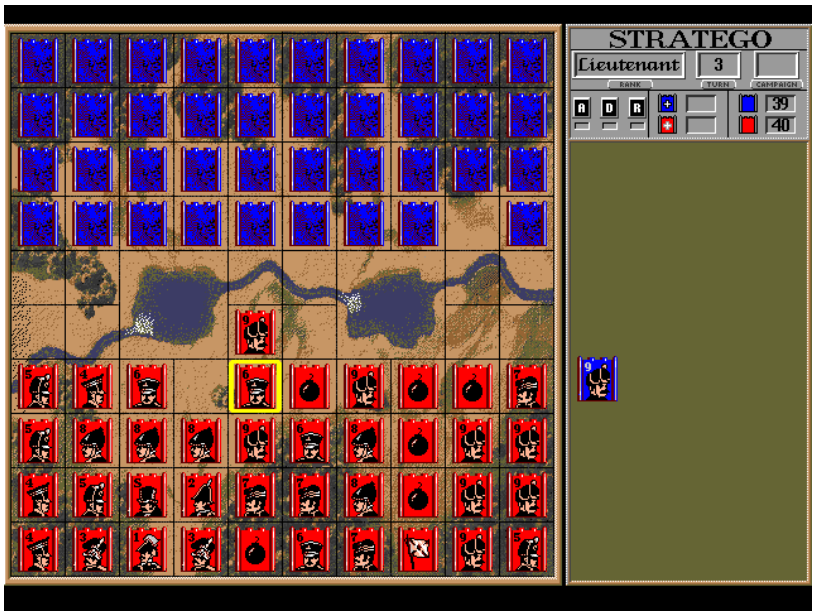


Figure 3.2: Stratego Game board.

identity inferences. These embeddings form the base for all subsequent reasoning. A league-based training method trains and evaluates MARQ agents simultaneously.

3.3 MARQ Policy Definition

Policy optimization in reinforcement learning improves an agent's strategy to maximize total reward [41]. Games with hidden information and multiple players demand policies that perform well against fixed opponents while also handling uncertainty and shifting strategies from other agents. MARQ connects belief state reasoning with value-based decisions to meet these demands.

Policy learning in MARQ serves two goals. The first is winning games through tactical choices. The second is improving the quality of the embeddings that the AAREN module produces. The two objectives reinforce each other: better embeddings sharpen decision-making, and good decisions reveal opponent information that in turn refines the embeddings.

Table 3.1: Key symbols and descriptions.

Symbol	**Description**	**Example (MARQ Framework)**
$\pi_i(a\|I_i)$	Policy for agent i choosing action a given information set I_i	Probability of moving a piece forward given current visible board state and belief about opponent pieces
$\mathcal{B}_t$	Implicit belief representation at time t	Probability distribution over possible identities and positions of hidden opponent pieces inferred by AAREN
$v_i^{\pi}(h, \mathcal{B}_t)$	Value function for agent i following policy π given history h and belief $\mathcal{B}_t$	Expected reward for current board position considering uncertainty about opponent setup
$Q_{\text{network}}(s', a)$	Deep Q-Network approximation of action-value function	Neural network estimate of value for attacking with a suspected Marshal vs opponent piece
$R^{\text{capture}}i, t+1$	Immediate reward from piece captures weighted by piece values	+9 points for capturing opponent Marshal, -1 for losing a Scout
$O = \{o_1, \ldots, o_t\}$	Observation sequence processed by AAREN attention mechanism	Sequence of board states, moves, and attack outcomes throughout the game
α_i	Attention weights for historical observations in AAREN	Higher weight on turn when opponent's Marshal was revealed, lower on routine Scout moves
π^*	Nash equilibrium policy profile	Optimal mixed strategy that cannot be exploited by opponent strategy changes
$Q_{eval}(\mathcal{B})$	Embedding quality evaluator output	Confidence score for learned representation accuracy
$H(\pi)$	Policy entropy measure	Ensures minimum unpredictability to prevent exploitation
ϵ	Exploration rate for agent behavior	Controls exploration vs exploitation balance in multi-agent learning
γ	Discount factor for future rewards	Importance of long-term vs immediate rewards in value estimation

3.3.1 Core Policy Framework

A policy in the MARQ framework, denoted $\pi_i(a|I_i)$, defines a probability distribution over actions for agent i conditioned on its current information set I_i. In perfect information games, policies operate on complete game states. MARQ policies do not have that luxury. They reason under uncertainty about the true state of the environment. The information set I_i contains all observations available to the agent: visible board positions, revealed piece identities, and the history of observed movements and combat outcomes.

The core difficulty is that agents must act without knowing the opponent's private information. In Stratego, this means the rank and position of every unrevealed opponent piece remain unknown. The policy must incorporate estimates of the opponent's pieces into its decision process and select actions that perform well across many possible hidden states.

MARQ addresses this by using implicit belief representations $\mathcal{B}_t$ rather than enumerating every possible game state. The AAREN module learns and updates these representations as new information arrives. The policy takes the following form:

$$\pi_i(a|I_i, \mathcal{B}_t) = P(\text{action} = a \mid \text{information set } I_i, \text{implicit belief } \mathcal{B}_t) \tag{3.2}$$

The policy learns to weight actions based on their expected value across the inferred distribution of opponent configurations rather than optimizing for a single specific setup. Playing well requires this kind of reasoning about hidden states.

3.3.2 Implicit Representation Optimization

The quality of the implicit belief representation directly affects decision quality. Inaccurate inferences produce poorly calibrated action values. MARQ therefore treats representation accuracy as an auxiliary training objective and uses revealed piece identities as supervised labels for embedding refinement.

When combat reveals the piece identities, the system calculates the prediction error using Kullback-Leibler (KL) divergence.

$$\mathcal{E}_{\text{pred}}(t) = \sum_{p \in \text{Pieces}_{\text{revealed}}(t)} D_{\text{KL}}\left(\text{Representation}_t(p) \| \delta_{s_p^*}\right) \tag{3.3}$$

Here, $\delta_{s_p^*}$ denotes the Dirac delta distribution concentrated on the true piece type. The error quantifies how far the predicted distribution lies from the actual identity.

This prediction error becomes a representation accuracy reward signal. It encourages the policy to develop embeddings that better predict actual piece configurations:

$$R_{\text{accuracy}}(t) = -\mathcal{E}_{\text{pred}}(t) = -\sum_{p \in \text{Pieces}_{\text{revealed}}(t)} \left[\text{Representation}_t(p = s_p^*) > 0\right] \cdot \log \text{Representation}_t(p = s_p^*) \tag{3.4}$$

The optimal policy maximizes the joint objective of game outcomes and representation accuracy:

$$\pi_i^* = \arg\max_{\pi_i} \mathbb{E}_{\tau \sim \pi_i} \left[\sum_{t=0}^{T} \gamma^t \left(R_{\text{game}}(t) + \eta \cdot R_{\text{accuracy}}(t) \right) \right] \tag{3.5}$$

The hyperparameter $\eta \in \mathbb{R}^+$ controls the relative importance of representation accuracy versus immediate game rewards.

The value function in MARQ extends the standard Bellman equation to account for belief dynamics. It expresses the expected cumulative reward as a function of the current information set and implicit belief.

$$v_i^\pi(I, \mathcal{B}_t) = \mathbb{E}_\pi \left[R_{i,t+1}^{\text{capture}} + \lambda_1 R_{i,t+1}^{\text{info}} + \lambda_2 R_{i,t+1}^{\text{position}} + \gamma v_i^\pi(I_{t+1}, \mathcal{B}_{t+1}) \mid I_t = I, \mathcal{B}_t \right] \tag{3.6}$$

Three strategic components make up the reward function. The capture reward R^{capture} reflects material changes. The information reward R^{info} quantifies the value of information revelation. The position reward R^{position} measures territorial control.

MARQ training aims to reach a Nash equilibrium. At equilibrium, no player can improve their expected payoff through unilateral deviation. This stability ensures that the policy remains robust against adversarial exploitation.

$$v_i(\pi^*) = \max_{\pi_i} v_i(\pi_i, \pi_{-i}^*) \tag{3.7}$$

Finding exact Nash equilibria in large imperfect information games is computationally intractable. MARQ instead seeks approximate equilibria through self-play training with diverse opponent populations.

3.4 Deep Q-Network Architecture

The Deep Q-Networks in MARQ operate on implicit belief representations rather than complete game states [41]. The network approximates Q-values over probability distributions of opponent configurations, $\mathcal{B}_t$:

$$Q(s, a) = \mathbb{E}_{s' \sim \mathcal{S}_t \subseteq \mathcal{B}_t}[Q_{network}(s', a)]$$

A standard replay buffer stores experience. Transitions $(\mathcal{B}_t, a_t, r_t, \mathcal{B}_{t+1})$ are sampled uniformly to train the value function.

3.4.1 Rainbow DQN Components

The implementation incorporates several enhancements from the Rainbow DQN architecture [20] to improve stability and sample efficiency.

Distributional RL (Categorical DQN). Categorical DQN (C51) models the value distribution directly. It captures the inherent uncertainty of the game by approximating the distribution with 51 atoms spanning the range from V_{min} to V_{max}.

$$Z_\theta(s, a) = \sum_{i=1}^{N} p_i(s, a)\delta_{z_i} \tag{3.8}$$

Training minimizes the Kullback-Leibler divergence between the predicted distribution and the projected target distribution:

$$D_{KL}(\Phi\hat{\mathcal{T}}Z_{\theta'}||Z_\theta) \tag{3.9}$$

where Φ is the projection operator that maps the Bellman update onto the fixed support atoms.

Noisy Networks for Exploration. Noisy Linear layers inject learned noise into the network weights. The agent maintains an unpredictable strategy without a separate exploration schedule, and the network itself optimizes the degree of exploration required for each state.

$$y = (\mu_w + \sigma_w \odot \epsilon_w)x + (\mu_b + \sigma_b \odot \epsilon_b) \tag{3.10}$$

Dueling Network Architecture. The network separates state value estimation $V(s)$ from advantage estimation $A(s, a)$ using two streams that share a common convolutional feature extractor:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right) \tag{3.11}$$

This decomposition improves policy evaluation when many actions carry similar value.

An exploration bonus augments the Q-values. The bonus is based on the uncertainty of the inferred representation, following an optimism-in-the-face-of-uncertainty approach.

$$Q_{augmented}(s, a) = Q_{base}(s, a) + \lambda \cdot U(a) \tag{3.12}$$

Here $U(a)$ represents the entropy of the learned distribution at the target location. This term encourages the agent to investigate unknown opponent pieces.

3.4.2 Training Stability via Regularization

The Ataraxos system reached superhuman performance in no-press Diplomacy, and its designers found that training stability matters as much as the algorithm itself [39]. Deep reinforcement learning agents face two recurring problems: wasting updates on low-information experiences, and cycling between strategies without settling on a robust one.

Advantage Filtering for Sample Efficiency. Advantage filtering concentrates learning on the most informative transitions [44]. In reinforcement learning, the temporal-difference (TD) error measures the gap between predicted values and outcomes. A standard replay buffer samples past transitions at random, treating every move equally. Advantage filtering breaks this uniformity. It samples the buffer and retains only the transitions with the largest TD errors:

$$\mathcal{B}_{filtered} = \mathrm{Top}_{n/k} \left(\mathcal{B}_{sampled}, |\delta_i| \right) \tag{3.13}$$

where $\delta_i = r_i + \gamma^n V(s'_i) - V(s_i)$ is the n-step TD error for transition i.

Dynamic Damping with Power-Law Annealing. Strategy cycling is addressed through two regularization terms in the loss function, both based on Kullback-Leibler divergence:

$$\mathcal{L}_{total} = \mathcal{L}_{C51} + \alpha(t) \cdot D_{KL}(\pi \| \pi_{uniform}) + \beta(t) \cdot D_{KL}(\pi_{target} \| \pi) \tag{3.14}$$

The coefficients follow power-law schedules with opposing dynamics:

$$\alpha(t) = \alpha_0 \cdot (1 - t/T)^p \tag{3.15}$$

$$\beta(t) = \beta_0 + (\beta_T - \beta_0) \cdot (t/T)^p \tag{3.16}$$

where t is the current episode, T is the total training duration, and $p = 2$ produces quadratic curves. Early in training, when uncertainty is highest, the agent explores aggressively. As the policy matures, the schedule shifts weight toward stability.

3.5 AAREN and Learned Embedding System

The AAREN module (Attention as Recurrent Neural Network) [18] serves as the memory and history system within MARQ. It does not maintain simple counts of piece identities. Instead, it produces learned embeddings from opponent moves, and the agent interprets these embeddings through training. AAREN handles long sequences in Stratego. It retains details over hundreds of moves while avoiding the gradient problems that degrade older models such as LSTMs. An attention mechanism lets AAREN look back across the full sequence of moves at once [18].

3.5.1 Recurrent Attention Computation

AAREN implements an attention-based recurrent computation that maintains a state triplet (a_t, c_t, m_t) across the observation sequence. Traditional recurrent neural networks suffer from vanishing gradients over long sequences. AAREN avoids this entirely. It provides direct access to the full history through attention weights while maintaining $O(1)$ inference complexity per timestep.

Given a sequence of observations $O = \{o_1, \ldots, o_t\}$ where each o_i encapsulates the visible board state and public action outcomes, the AAREN cell first projects each observation into key and value representations:

$$k_t = W_k(o_t), \quad v_t = W_v(o_t) \tag{3.17}$$

The projection matrices W_k and W_v are learned during training. The key k_t determines how relevant the current observation is for belief updates. The value v_t encodes the informational content to be integrated into the belief representation. The attention score s_t measures the relevance of the current observation relative to a learned query vector q:

$$s_t = q^T k_t \tag{3.18}$$

The weighted value accumulator a_t aggregates value representations across the sequence:

$$a_t = a_{t-1} \exp(m_{t-1} - m_t) + v_t \exp(s_t - m_t) \tag{3.19}$$

The term $\exp(m_{t-1} - m_t)$ rescales the previous accumulator when a new maximum appears. The term $\exp(s_t - m_t)$ computes the relative attention weight for the current observation. The normalization constant $c_t = c_{t-1} \exp(m_{t-1} - m_t) + \exp(s_t - m_t)$ ensures that the effective attention weights sum to unity. The output $h_t = a_t / c_t$ is an attention-weighted aggregation of all observations up to time t.

3.5.2 Piece Type Inference

AAREN infers piece types from action sequences. Each action is encoded as a 24-dimensional feature vector capturing movement patterns, contextual information, and game state features. The network outputs a probability distribution over piece types:

$$P(\text{piece\_type} = t \mid \text{action\_sequence}) = \text{softmax}(W_{out} \cdot h_T + b_{out})_t \tag{3.20}$$

The network combines these embeddings with game rules and the board state to improve decisions under uncertainty.

An evaluation system monitors embedding quality and drives improvement. A neural network evaluator takes embeddings as input and outputs quality scores $Q_{eval}(\mathcal{E}) = \text{MLP}(\mathcal{E}; \theta_{eval})$. Training compares predictions against true piece identities:

$$R_{quality}(\mathcal{B}, g) = \alpha \cdot \mathcal{B}[g] - \beta \cdot |V_{pred} - V_{actual}| - \gamma \cdot \text{distance\_penalty} \tag{3.21}$$

The system decides when additional information gathering is worth the cost through uncertainty assessment:

$$\text{NeedMoreInfo}(\mathcal{B}) = \mathbb{1}\{H(\mathcal{B}) > \theta_H\} \cdot \mathbb{1}\{Q_{eval}(\mathcal{B}) < \theta_Q\} \tag{3.22}$$

3.5.3 Embedding Updates on Piece Revelation

When a battle reveals a piece's true type, the AAREN embeddings synchronize with ground truth. The inferred distribution at the revealed position collapses to certainty:

$$\text{Representation}_{pos}[t] = \begin{cases} 1.0 & \text{if } t = g \\ 0.0 & \text{otherwise} \end{cases} \tag{3.23}$$

The system increments the revealed piece count and adjusts remaining beliefs to respect piece count constraints. For unrevealed positions $p \neq pos$:

$$\mathcal{B}_p[t] \leftarrow \mathcal{B}_p[t] \cdot \frac{\text{max\_count}[t] - \text{revealed\_count}[t]}{\text{remaining\_count}[t]} \tag{3.24}$$

For example, when the single Marshal is revealed, the probability of any other piece being a Marshal drops to zero.

3.5.4 Hindsight-Aided Belief Refinement

The AAREN module provides inference during gameplay. The Hindsight-Aided Belief Refinement (HABR) module adds a second channel: learning from verified information. When combat reveals a piece, HABR applies retrospective learning across the entire history of that piece, using the ground truth as a supervisory signal for past predictions.

Information Gain Reward. The Information Gain reward quantifies how much an observation reduces uncertainty about opponent pieces. Let $H(P)$ denote the Shannon entropy of a probability distribution P. The KL divergence from a uniform prior U over N piece types is:

$$D_{KL}(P \parallel U) = \log(N) - H(P) \tag{3.25}$$

The Information Gain reward measures the change in this divergence between consecutive representation updates:

$$R_{gain}(t) = D_{KL}(\text{Rep}_{t-1} \parallel U) - D_{KL}(\text{Rep}_t \parallel U) \tag{3.26}$$

Substituting the divergence formula yields:

$$R_{gain}(t) = (\log(N) - H(\text{Rep}_{t-1})) - (\log(N) - H(\text{Rep}_t)) = H(\text{Rep}_{t-1}) - H(\text{Rep}_t) \tag{3.27}$$

R_{gain} equals the reduction in entropy. A positive reward occurs only when the observation at time t increases certainty about piece identity relative to $t-1$. This aligns the agent's incentives with active information gathering.

Sinkhorn Constraint Normalization. Independent belief updates for each piece position can produce inconsistent global predictions. Without piece count constraints, the model might assign high probability to multiple pieces being the single Marshal. This phenomenon is called identity inflation. It violates the physical constraints of the game.

Sinkhorn normalization corrects this through iterative proportional fitting. Given a representation matrix $M \in \mathbb{R}^{P \times S}$ where P is the number of tracked pieces and S is the number of piece types, two constraints must hold. The row constraint $\sum_s \text{Rep}(p, s) = 1$ ensures each piece has exactly one identity. The column constraint $\sum_p \text{Rep}(p, s) = Count(s)$ ensures the total predicted count of each piece type matches the remaining count.

The Sinkhorn algorithm alternates between row and column normalization until convergence. The approach is differentiable, so gradients pass through it. When piece A is confirmed as the Spy, the Sinkhorn layer automatically redistributes probability across all other pieces, creating a shared update across the entire inferred state.

3.6 Multi-Agent Learning and Strategy Mixing

3.6.1 League-Based Training Architecture

The league maintains a changing pool of agents so that the model encounters many different strategies [35]. The main agent is the best current policy under training. The historical pool stores earlier versions of the main agent, each representing a different stage of development.

Opponents are sampled during training according to a fixed probability distribution.

$$P(\text{opponent}) = \begin{cases} 0.5 & \text{Main Agent (Self-Play)} \\ 0.5 & \text{Uniform(Historical Pool)} \end{cases} \tag{3.28}$$

This prevents the agent from cycling through the same strategies repeatedly. It must defeat all previous versions of itself.

3.6.2 Strategy Mixing for Exploitation Resistance

Deterministic strategies in extensive-form imperfect information games are inherently suboptimal [35, 42]. An adversary who knows a deterministic policy reduces the game to a perfect information setting, eliminating the strategic uncertainty that defines games like Stratego. MARQ counters this through two complementary mechanisms.

At the action selection level, Noisy Linear layers inject learned, state-dependent noise into the policy. The network learns the appropriate degree of randomization for each game state rather than relying on fixed exploration schedules such as ϵ-greedy.

At the strategy level, diverse opponent exposure during training achieves mixing. The league system maintains a population of historical agent checkpoints, each representing a different strategic approach discovered during training. Action randomness combined with strategy variety produces policies that are unpredictable and resistant to exploitation.

3.6.3 Multi-Dimensional Reward Structure

The reward function $R(s, a, s')$ guides learning in Stratego's sparse-reward environment [15]:

$$R_{total} = R_{move} + R_{capture} + R_{tactical} + R_{info} \tag{3.29}$$

The move reward R_{move} gives small bonuses for advancing pieces (approximately +0.05) to encourage active play. The capture reward $R_{capture}$ scales by the rank of the captured piece according to $0.15 \times \frac{\text{Rank}}{11.0}$. The tactical reward $R_{tactical}$ assigns specific bonuses for key strategic captures: +0.5 for Miner vs Bomb defusals and +1.0 for Spy vs Marshal assassinations. The information reward R_{info} provides +0.3 for revealing high-value opponent pieces.

The framework uses *Reward Annealing* to transition from tactical learning to strategic gameplay. Dense shaping rewards are scaled down as the training curriculum advances. In later training phases, these auxiliary signals are completely removed, and the agent relies exclusively on sparse, terminal win/loss rewards. This progression prevents the agent from developing sub-optimal behaviors driven by short-term shaping incentives.

3.7 Specialized Agent Architectures

3.7.1 Setup Agent via Distributional RL

A specialized SetupAgent handles the game's initial setup. It learns optimal piece configurations by treating the placement phase as a sequential decision process. The state representation is a 3-channel input tensor: one channel encodes the current board state with already-placed pieces, a second provides a valid positions mask indicating legal placement locations, and a third identifies which piece type is currently being placed.

The agent outputs a distributional Q-value for each of the 100 board positions, mapping $Q(s, pos)$ to a distribution over $[V_{min}, V_{max}]$. This distributional approach captures the inherent uncertainty in setup quality, since the true value of a placement depends on the unknown opponent configuration.

An exploitability critic network penalizes predictable placements to prevent static, easily countered setups:

$$\mathcal{L}_{total} = \mathcal{L}_{C51} + \lambda \cdot \mathcal{L}_{predictability} \tag{3.30}$$

3.7.2 Information Set Monte Carlo Tree Search (IS-MCTS)

For high-stakes decisions, especially in the endgame, MARQ uses an Information Set MCTS agent [12] that combines search with learned value functions. The search operates in three phases.

In the determinization phase, the agent samples a consistent concrete world state s_{det} from the history-based beliefs according to $s_{det} \sim P(S|\mathcal{B}_t)$. This converts the imperfect information game into a perfect information instance for the duration of the simulation.

In the tree traversal phase, a UCB1 variant navigates the search tree to balance exploration and exploitation of move sequences. Nodes represent information sets rather than individual states, so the tree can be shared across multiple determinizations.

In the leaf evaluation phase, terminal or unexpanded nodes are evaluated using the Rainbow DQN value function instead of random rollouts:

$$V(s_{leaf}) \approx Q_{DRQN}(s_{leaf}, a_{best}) \tag{3.31}$$

3.8 Game Environment and State Management

The Stratego environment provides the foundation for agent training and evaluation. It manages the complex dynamics of imperfect information.

3.8.1 Information Set Management

The environment maintains distinct representations for each information context:

$$\mathcal{I}_{player} = \{s : s \text{ is consistent with player observations}\} \tag{3.32}$$

Each player receives only the subset of game state information available through its observation function.

3.8.2 Temporal State Evolution

Game states evolve according to an update function.

$$s_{t+1} = \text{EnvStep}(s_t, a_t^1, a_t^2, \theta_t) \tag{3.33}$$

Here θ_t represents stochastic elements such as hidden information revelation and time limits.

3.8.3 Valid Action Filtering

The environment enforces game rules through valid action constraints:

$$\mathcal{A}_{valid}(s, player) = \{a : \text{IsLegalMove}(a, s, player) \wedge \text{RespectsGameRules}(a, s)\} \tag{3.34}$$

Only legal moves consistent with Stratego's rules and piece capabilities are available to the agent.

3.8.4 Piece Setup and Initialization

Setup policies govern strategic piece placement.

$$\pi_{setup}(\text{placement}|player) = \text{NeuralNetworkSetup}(\text{strategic\_features}, \theta_{setup}) \tag{3.35}$$

Strategic features include positional value maps, piece interaction potentials, and defensive vulnerabilities. The initial placement of pieces is a high-impact decision that shapes the remainder of the game [14].

3.9 MARQ Move Selection and Decision-Making

The MARQ framework's architectural components converge in the real-time move selection process. Algorithm 1 details this procedure, which integrates learned positional priors with tactical search to optimize behavior in adversarial environments.

Algorithm 1 MARQ: Decision-making and Move Selection

Input: Public Observations O, Board State S, DQN weights θ, AAREN weights ϕ, Temperature T
Output: Optimal Action a^*

$\mathcal{H}_t \leftarrow \text{UpdateAggregation}(O, \phi)$
$\mathcal{B}_t \leftarrow \text{GetSoftmaxBeliefs}(\mathcal{H}_t)$
$X_t \leftarrow \text{Concatenate}(S, \mathcal{H}_t)$
$Q(s, \cdot) \leftarrow \text{RainbowInference}(X_t, \theta)$
$\mathcal{A} \leftarrow \text{LegalMoves}(S)$
for all $a_i \in \mathcal{A}$ **do**
  if $\text{IsAttack}(a_i, S)$ **then**
    $target\_pos \leftarrow \text{TargetSquare}(a_i)$
    $P \leftarrow \mathcal{B}_t(target\_pos)$
    $V_{exp} \leftarrow \sum_{r \in \mathcal{R}} P(r) \cdot \mathbb{U}(\text{Attacker}, r)$
    $V_{final}(a_i) \leftarrow \lambda Q(s, a_i) + (1 - \lambda) V_{exp}$
  else
    $V_{final}(a_i) \leftarrow Q(s, a_i)$
  end if
end for
if Training/Self-Play and $T > 0$ **then**
  $\pi(a_i) \leftarrow \frac{\exp(V_{final}(a_i)/T)}{\sum_{a \in \mathcal{A}} \exp(V_{final}(a)/T)}$
  return $a^* \sim \pi(\cdot)$
else
  return $a^* \leftarrow \arg\max_{a_i \in \mathcal{A}} V_{final}(a_i)$
end if

3.9.1 Historical Context Update

The decision process begins with the AAREN module's recurrent attention system. As game observations arrive, the history aggregator updates its internal state. The agent's current embedding

thus reflects the opponent's cumulative behavioral patterns, not just the immediate board state.

3.9.2 Probabilistic Belief Extraction

After updating the history, the system extracts softmax probability distributions for hidden pieces. These piece-identity predictions are generated for every square on the board. They represent the agent's current belief about the hidden ranks of unrevealed opponent pieces.

3.9.3 Information Set Tensorization

The visible board state and the learned AAREN embeddings are concatenated to form a high-dimensional information set representation. In the current implementation, this produces a 79-channel input tensor (15 physical board features + 64 history embedding channels), which encapsulates both the physical layout and the inferred hidden context.

3.9.4 Value Distribution Estimation

The Rainbow DQN's convolutional backbone and C51 distributional head process the 79-channel tensor. This step estimates the Q-value distribution over 51 atoms for all possible actions. The distribution captures the multi-modal uncertainty inherent in the state.

3.9.5 Action-Space Filtering

Legal move masking ensures rule compliance and efficiency. The Q-network's output is filtered against the set of valid moves $\mathcal{A}$ defined by Stratego's rules. Illegal actions cannot be selected.

3.9.6 Expectamax Search and Tactical Summation

Standard adversarial search algorithms like Minimax assume an opponent who plays optimally to minimize the agent's utility. Many environments involve stochastic elements or hidden information that are better modeled as probabilistic transitions [41, 10]. Algorithms addressing this class of problems introduce "Chance Nodes" into the game tree.

In Stratego, the identity of an opponent piece encountered during an attack is a stochastic variable. The Expectamax value is an exact summation over the rank probabilities provided by the

belief extractor:

$$\mathbb{E}[V] = \sum_{r \in \mathcal{R}} P(\text{target is rank } r) \cdot \mathbb{U}(\text{outcome vs } r) \tag{3.36}$$

The agent can pursue tactical aggression when the probability of a successful capture outweighs the risk of piece loss. The decision is grounded in the AAREN's inference history.

3.9.7 Stochastic Action Selection (Temperature Scaling)

Rigid, exploitable behavior loops must be avoided during self-play and evaluation. The final action selection step uses Temperature Scaling (Boltzmann Options). Instead of choosing the action with the highest expected value via a hard *argmax*, the agent samples from a stochastic softmax distribution parameterized by a temperature variable T. The probability of selecting action a_i follows the Boltzmann distribution over the action values V_{final}:

$$\pi(a_i) = \frac{\exp\left(V_{final}(a_i)/T\right)}{\sum_{a \in \mathcal{A}} \exp\left(V_{final}(a)/T\right)} \tag{3.37}$$

As T approaches zero, the selection becomes deterministically greedy. When T increases, near-optimal actions receive proportionally higher selection probabilities, injecting controlled variance. This stochasticity serves two purposes: it generates a diverse and unpredictable curriculum roster during league training, and it prevents tactical stagnation within recurring positional maneuvers.

3.10 Generalizability

The MARQ framework provides mathematical stability in adversarial settings and applicability to real-world domains beyond gaming. This section presents the convergence guarantees of the training methodology and the potential for transferring learned representations to non-gaming environments.

3.10.1 Mathematical Convergence Properties

MARQ provides theoretical guarantees for convergence and performance under specific conditions.

Incremental Learning Bounds

The algorithm bounds value function estimation error as follows:

$$\|V_t - V^*\|_\infty \leq \frac{C}{\sqrt{t}} + \epsilon_{approx} \tag{3.38}$$

where C depends on reward bounds and the discount factor, and ϵ_{approx} represents function approximation error.

Implicit Representation Accuracy Bounds

Representation accuracy improves with additional observations:

$$\mathbb{E}[\text{Accuracy}(\mathcal{B}_t)] \geq 1 - \exp(-\alpha \cdot N_{obs}(t)) \tag{3.39}$$

where $N_{obs}(t)$ is the cumulative number of informative observations and α is a positive constant.

Strategy Convergence in Self-Play

The league training methodology converges to ϵ-Nash equilibria under standard monotonic improvement assumptions:

$$\lim_{T \to \infty} \|\pi_T - \pi^*\| \leq \epsilon \tag{3.40}$$

for some $\epsilon > 0$ depending on the exploration parameters and opponent diversity. Hierarchical best-response maintenance, main-pool promotion thresholds, and diversity-driven exploration drive this convergence.

3.10.2 Applied Generalizability

The MARQ framework was developed for Stratego, but its architecture applies to broader problems involving partial observability and adversarial decision-making.

Inference in Degraded Environments

The occlusion-based curriculum combined with AAREN history aggregation provides a method for training agents to operate in degraded environments. In autonomous drone applications, sensors may fail or be obstructed by smoke or weather. MARQ's ability to maintain an implicit belief state allows continued navigation and objective completion despite sensor loss.

Signal Intelligence and Pattern Discovery

In adversarial signal environments such as electronic warfare, implicit representation learning can discover latent identities in high-dimensional signal data. The network treats signal history as an action sequence and learns to categorize threats without explicit labeling of every transmission.

Industrial Automation and Predictive Maintenance

MARQ's architecture can integrate sparse sensor readings over time to predict hidden anomalies or maintenance needs in complex manufacturing pipelines. This enables proactive adjustments before system failure occurs.

These convergence properties and the transferability of the learned embeddings support confidence in the framework's ability to handle high-stakes gameplay and transition to practical applications.

3.11 Algorithmic Evolution and Architecture Ablations

The final MARQ architecture emerged from systematic experimentation and the refinement of several reinforcement learning components. Multiple established algorithms were evaluated and replaced because of performance bottlenecks or insufficient strategic depth in the Stratego domain.

Value Optimization and Exploration. Early iterations used a normal DQN with ϵ-greedy exploration. This approach proved inadequate for Stratego's sparse rewards and led to early stagnation. The implementation was upgraded to a Rainbow DQN architecture incorporating Noisy Networks for state-dependent exploration. The standard double-DQN configuration was then replaced with a Dueling Head architecture, which improved value estimation accuracy in states where multiple actions carry similar utility. To mitigate overfitting to deterministic opponent policies during self-play, the system also employs *Boltzmann Exploration* (Temperature Scaling). The opponent's action selection uses a temperature-scaled softmax over Q-values, injecting controlled stochasticity to produce a more robust training curriculum.

History and Memory Representation. Initial attempts to manage hidden information relied on explicit Probabilistic Belief States. These states introduced prohibitive computational complexity and memory overhead. They required $O(N^2)$ updates that throttled training throughput. The framework moved to an implicit belief system powered by the AAREN architecture. Long Short-Term Memory cells for trajectory embeddings were also replaced by AAREN's attention-based recurrence, which provided more stable gradient flow over long sequences and better handled the non-Markovian nature of piece identity inference.

Architectural Refinements. The convolutional backbone underwent several iterations, moving from standard deep convolutional stacks to a dedicated ResNet architecture. Testing identified 6 Residual Blocks as the optimal depth. This configuration provides full receptive field coverage of the 10x10 board while maintaining low inference latency. The optimizer was transitioned from standard ADAM to ADAMW to improve weight decay and policy loss stability.

Strategic Initialization and Efficiency. The piece placement phase was initially modeled using an agentic SetupAgent trained through distributional reinforcement learning. This approach introduced exploitability risks and operational overhead. The project transitioned to a non-agentic, fixed Heuristic Setup Agent with randomized flag positioning. The change resolved the exploitability concern and provided substantial computational savings. The agentic setup required approximately 2 seconds per episode; the heuristic method completes in under 100 milliseconds. This optimization effectively halved the total training compute requirement.

Chapter 4

Methodology

This chapter describes how the MARQ framework was tested for its ability to play games with hidden information. A competitive system was built and then evaluated on win-rate against human players and adaptability across different games. The following sections detail each step.

4.1 Project Planning

This research followed a plan covering participant selection and experimental setup. A rapid prototyping method guided the design and construction of the MARQ framework. Development ran from September 2025 to February 2026 and included building the game environment and training the agent. User testing took place in March 2026. Data analysis followed in April 2026.

4.1.1 Community Introduction to Stratego

Participants were first introduced to Stratego because none of them knew the rules or strategies. Stratego is a niche game, and experienced players were difficult to recruit for testing. Teaching everyone the game from scratch meant that both the humans and the AI started from the same baseline, producing a fairer comparison.

Twenty participants were selected based on their availability and experience with strategy games. The selection criteria targeted people from different backgrounds who enjoyed strategic thinking but had no prior Stratego experience. This ensured they had the cognitive skills needed to play well

while starting fresh with the specific game, maintaining a consistent starting point for the study.

4.1.2 Experimental Procedures

One-on-one matches between human participants and the MARQ framework will be conducted on-site using a single device. Each participant will receive a checklist outlining the procedures for interacting with the game to ensure consistency.

Each session will last one hour, during which participants must complete at least two games against the MARQ framework. Additional games may be played if time permits. The recorded statistics will include the winner, total game duration, total moves per player, and the number and type of remaining pieces for both players. Immediately after the session, each participant will complete an evaluation form providing open-ended feedback on their experience.

4.2 System Development

System development consists of two components: the game environment and the MARQ framework. The game environment serves as the interface for human players and the world in which the trained agent operates. The MARQ framework provides the trained agent that competes against human players.

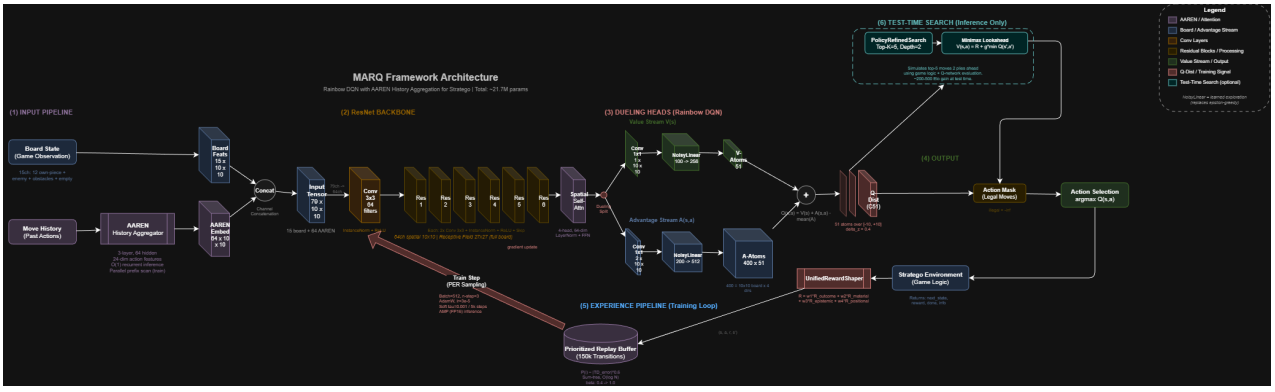


Figure 4.1: MARQ framework architecture.

4.2.1 Piece Value Computation

The reward function requires a quantitative assessment of piece value to guide learning. This section presents an analytical method for computing piece values based on probability theory, providing a deterministic alternative to empirical estimation through gameplay simulation.

Combat Dominance Score

The Combat Dominance Score (CDS) represents the probability that a given piece type defeats a uniformly random opponent piece. Let $\mathcal{P}$ denote the set of all piece types and n_i the count of piece type i in standard Stratego. The CDS for piece p is defined below.

$$\text{CDS}(p) = \frac{\sum_{i \in \mathcal{P}} \nVdash_{p \succ i} \cdot n_i}{N} \tag{4.1}$$

where $N = \sum_{i \in \mathcal{P}} n_i = 40$ is the total piece count, $\nVdash_{p \succ i}$ is an indicator function equal to 1 if piece p defeats piece i according to Stratego combat rules, and 0 otherwise. The combat rules specify that higher-ranked pieces defeat lower-ranked pieces, with specific exceptions. The Spy defeats the Marshal when attacking, and the Miner defeats Bombs.

Survival Rate

The Survival Rate (SR) measures the expected probability of a piece surviving an attack from a uniformly random attacker. Let $\mathcal{M} \subset \mathcal{P}$ denote the set of movable piece types (excluding Flag and Bomb). The SR for piece p is calculated as follows.

$$\text{SR}(p) = \frac{\sum_{a \in \mathcal{M}} \nVdash_{p \succeq a} \cdot n_a}{\sum_{a \in \mathcal{M}} n_a} \tag{4.2}$$

The indicator function equals 1 if piece p survives an attack from piece a.

Auxiliary Value Components

Certain pieces possess capabilities that extend beyond their combat rank. Table 4.2 enumerates these bonuses.

The Scout receives a mobility bonus of 1.0 due to its ability to move multiple squares per turn; all other movable pieces receive a base mobility of 0.5. Immovable pieces (Flag, Bomb) receive 0.

Table 4.1: Computed piece values with combat and survival metrics.

Piece	**Value**	**CDS**	**SR**
Marshal	8.4	0.825	0.939
General	8.0	0.800	0.939
Colonel	7.5	0.750	0.879
Major	6.7	0.675	0.788
Captain	5.7	0.575	0.667
Lieutenant	4.8	0.475	0.545
Miner	4.3	0.400	0.273
Sergeant	3.8	0.375	0.424
Bomb[†]	2.0	—	—
Spy	1.1	0.050	0.000
Scout	1.0	0.050	0.030
Flag[‡]	0.1	—	—

[†]Immovable defensive piece; eliminates any non-Miner attacker.

[‡]Immovable objective piece; capture results in game loss.

Table 4.2: Strategic ability bonuses for special-capability pieces.

Piece	**Bonus**	**Description**
Miner	2.0	Sole piece capable of removing Bombs
Spy	1.5	Defeats Marshal when initiating attack
Scout	1.0	Multi-square movement capability
Marshal	0.5	Highest combat rank

The scarcity of each piece type influences its strategic importance. The scarcity factor is computed using logarithmic scaling.

$$\mathrm{SF}(p) = \log\left(\frac{\max_{i\in\mathcal{P}} n_i}{n_p}\right) + 1 \tag{4.3}$$

These values inform the material reward component of the reward shaping function described in subsequent sections.

4.2.2 Using AAREN to Remember Game History

The primary challenge in Stratego involves decision-making under partial observability. Rather than utilizing explicit piece counts, the framework employs architectural patterns inspired by DeepNash [35]. The Attention-as-a-Recurrent-Neural-Network (AAREN) module aggregates history and generates embeddings derived from opponent maneuvers to facilitate piece identity inference.

To mitigate the "sparse death" problem, where a lack of observation history produces unin-

formative zero tensors, a learnable default embedding is integrated. This provides a consistent initialization for the network and maintains training stability during initial game states.

Action Feature Extraction. For each observed opponent action, AAREN extracts a 24-dimensional feature vector encoding movement features, position features, behavioral indicators, and temporal features.

4.3 Model Training

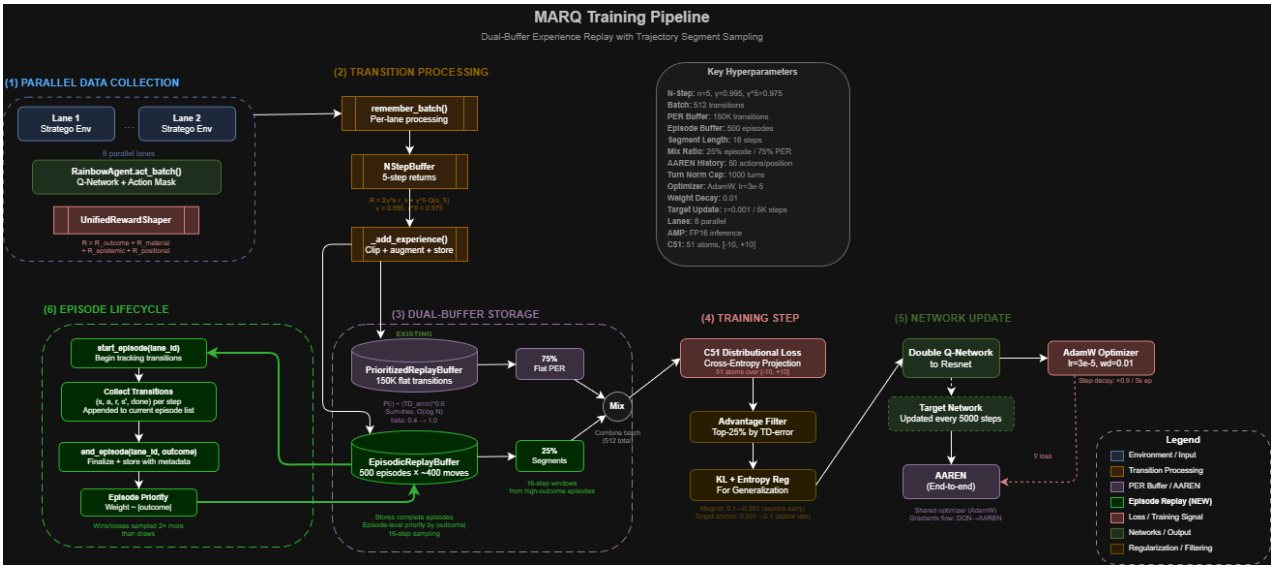


Figure 4.2: MARQ training architecture.

Training the MARQ agent is computationally intensive. The large state-action space and the need for accurate belief state representations over extended game horizons drive this cost.

Rainbow DQN Architecture. The MARQ system uses a Rainbow DQN framework with several architectural enhancements. Categorical DQN (C51) handles value distribution estimation. Noisy Networks drive exploration. A Dueling Network architecture decouples state values from action advantages. The noisy layers introduce state-dependent stochasticity, removing the need for manual epsilon-greedy scheduling.

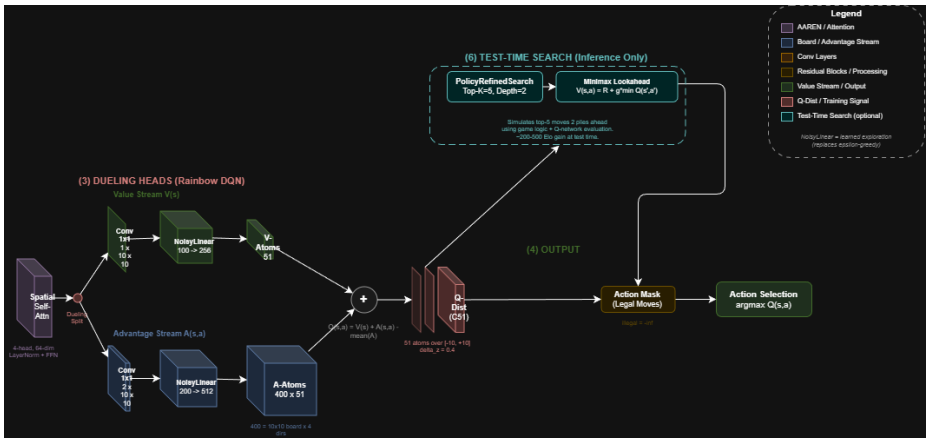


Figure 4.3: Dueling network heads.

Network Structure. The Q-network features a ResNet backbone for spatial feature extraction. An initial convolutional layer feeds into six residual blocks. This configuration provides a sufficient receptive field to evaluate global board state interactions.

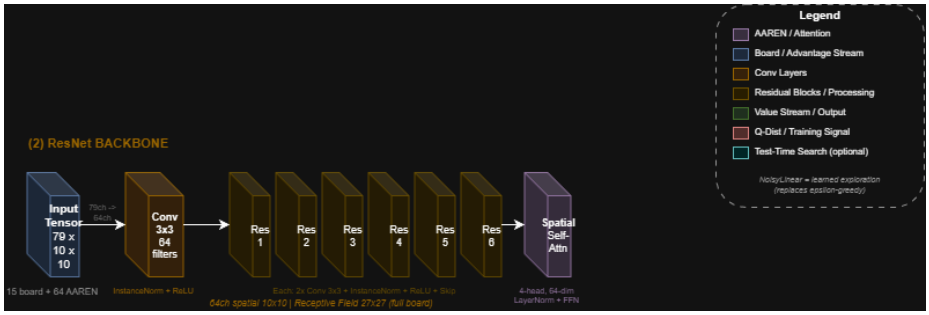


Figure 4.4: ResNet backbone.

A spatial self-attention layer follows the ResNet backbone. It captures relationships between pieces across the full board, even at long range. This layer feeds into the dueling heads, enabling the agent to develop strategies that depend on board-wide interactions.

The 79-channel state representation is partitioned into two functional sets, as summarized in Table 4.3.

The board feature channels provide deterministic knowledge of the agent's own piece positions and observed enemy locations. The AAREN embedding channels encode learned representations of hidden enemy piece identities based on observed behavior. These embeddings are dense learned vec-

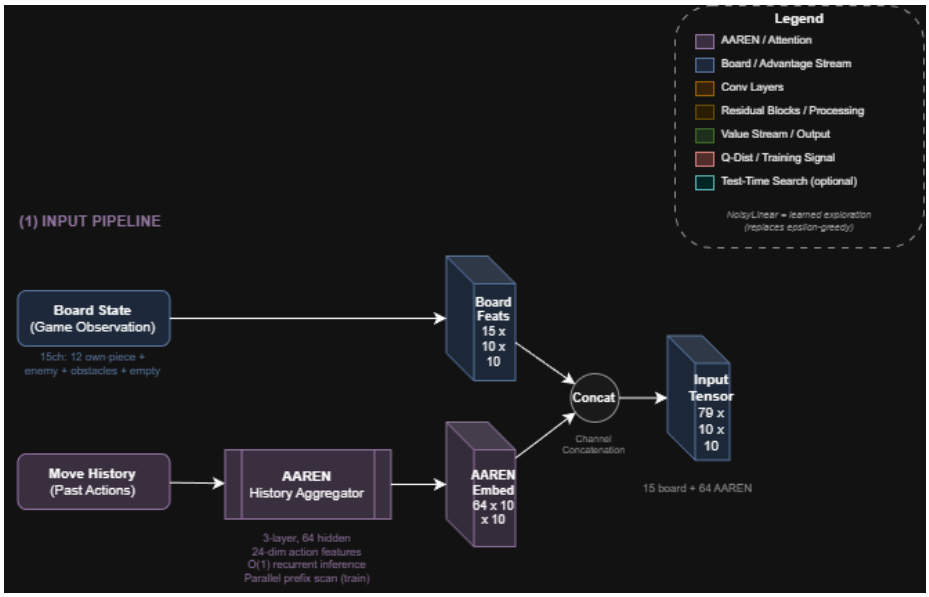


Figure 4.5: 79-channel input representation.

tors that the network interprets through end-to-end training, not explicit probability distributions. The AAREN embedding channels are populated in all training phases; the agent always receives embeddings from the history aggregator rather than ground truth one-hot encodings. The board feature channels carry different levels of enemy information depending on the observability setting. During full observability (Phase 1), the board tensor reveals all enemy piece identities, providing a strong signal in the piece-specific board channels. During partial observability phases, unrevealed enemy pieces appear only as a "hidden piece" presence marker (affecting Channel 12, the enemy mask), and the agent must rely on AAREN embeddings to infer their true identities. An earlier version suffered from a bug where the hidden piece marker was filtered out of Channel 12, blinding the network to the presence of enemy pieces. Correcting this restores spatial awareness while shifting the burden of identity inference to the AAREN module. The AAREN module uses 24-dimensional action feature vectors, 64 hidden units, and 3 layers to process action histories and output embedding vectors.

Training Hyperparameters. The training configuration specifies a learning rate $\alpha = 3 \times 10^{-5}$ with AdamW optimizer and weight decay $\lambda = 0.01$, and a discount factor $\gamma = 0.995$ to capture long-term strategic consequences. A batch size of 512 transitions and a replay buffer capacity of

Table 4.3: State representation input channels.

Channel	Category	Description
Board Features (Channels 0–14)		
0–11	Own Pieces	Binary spatial maps indicating positions of each piece type (Flag, Spy, Scout, Miner, Sergeant, Lieutenant, Captain, Major, Colonel, General, Marshal, Bomb)
12	Enemy Mask	Binary map indicating positions of all visible enemy pieces
13	Obstacles	Binary map of lake squares (impassable terrain)
14	Empty Squares	Binary map of unoccupied traversable positions
AAREN Embedding Features (Channels 15–78)		
15–78	Learned Embeddings	64-dimensional learned embedding vectors per board position, produced by the AAREN history aggregator from observed opponent action sequences. These embeddings encode implicit piece identity inference learned end-to-end with the agent.

150,000 transitions with prioritized sampling are used. Soft target network updates ($\tau = 0.001$) occur every 5,000 steps. Model updates run every 2 steps across 8 parallel lanes with 5-step returns for deeper credit assignment in long-horizon games. The AAREN history aggregator retains up to 50 actions per position in its training buffer to preserve early-game movement patterns across 200 to 500 move games. Turn normalization is aligned with the maximum game length of 1,000 turns. Horizontal flip augmentation exploits the game's symmetry to improve sample efficiency. Training proceeds for 35,000+ episodes with checkpoints saved every 1,000 episodes.

Computational Optimizations. Mixed-precision inference accelerates training throughput. During action selection, network forward passes operate in FP16 precision via `torch.amp.autocast`, reducing memory bandwidth requirements and leveraging Tensor Core acceleration on NVIDIA GPUs.

Integrating the recurrent AAREN module into the replay pipeline presents a significant bottleneck. Full sequential reconstruction of action histories during replay increases iteration time from 15s to 60s. To maintain high throughput, MARQ stores pre-computed embedding snapshots in the replay buffer. A "Gradient Bridge" preserves differentiability despite the detached snapshots: a differentiable epsilon-scaled zero-vector ($\epsilon \cdot W_{proj}(\mathbf{0})$) is added to the stored snapshots during replay. This preserves the 4x speedup while providing a gradient path from the DQN loss back

to AAREN's input projection layer. The resulting value-staleness is mitigated by AAREN's Dual Training Regime, which provides a fresh supervised signal during piece reveals.

Multi-Phase Curriculum. Training follows a multi-phase curriculum. Phase 1 establishes foundational rules under full observability. As win rates stabilize, the curriculum introduces mixed observability, requiring the agent to use AAREN-based inference before full information occlusion is enforced.

Phase 1 runs for 5,000 to 10,000 episodes against a progressively challenging opponent mix: initially random opponents, then basic heuristic agents, and finally a multi-factor heuristic opponent. Phase advancement requires $\geq$95% win rate against random opponents, $\geq$75% against the basic heuristic, and $\geq$60% against the strategic heuristic. All win-rate metrics are computed relative to decisive games (wins and losses) rather than total episodes to account for the high frequency of early-stage draws.

Phase 2 (Partial Observability) disables full observability and requires reliance on AAREN-generated embeddings for 5,000 to 10,000 episodes. The opponent distribution allocates 40% to frozen heuristic agents and 60% to the strategic heuristic. Phase advancement requires embedding-based inference accuracy $\geq$75% and an overall win rate $\geq$60% relative to decisive games.

Phase 3 (Self-Play) trains against a mixture of delayed agent copies (60%) and strategic heuristic opponents (40%) for 5,000 to 15,000 episodes to discover novel strategies while preventing policy collapse. Phase 4 (League Training) activates the full opponent distribution with 40% historical league agents, 30% strategic heuristic, 20% greedy agents, and 10% self-play. This phase also activates independent parallel-learning (MARL) for Agent 2, transitioning the opponent from a fixed policy pool to a continuously updating student agent. Phase 5 (Scenario Drills) introduces specialized board configurations every 1,000 episodes for targeted skill training.

4.3.1 Centralized Reward System and Defensive Normalization

The reward function drives agent behavior by transforming tactical events into learning signals. A centralized UnifiedRewardShaper consolidates all reward logic to ensure consistency and numerical stability.

Potential-Based Reward Shaping. PBRS provides the primary mechanism for dense learning signals. All intermediate shaping is unified into a single potential function $\Phi(s)$ that measures game progress rather than assigning additive rewards for individual events.

4.3.2 Action Space Reduction via Heuristic Filtering

The full action space in Stratego consists of 400 discrete actions, encoding each starting position on the 10×10 board paired with four cardinal directions. Many of these actions are illegal (moving off-board or into lakes) or strategically irrelevant at any given state. Presenting all 400 outputs to the network, even with invalid action masking, introduces noise as the network must distinguish between many low-value alternatives.

Heuristic Move Scoring. A heuristic pre-filter scores all legal moves and selects the top-k candidates ($k = 100$ by default) for consideration. The scoring function assigns points based on the criteria listed in Table 4.4.

Table 4.4: Heuristic scoring criteria.

Criterion	**Score**	**Rationale**
Attack (target is enemy)	+100	Captures are always strategically relevant
Forward advancement	+5/row	Territorial progress toward enemy base
Center control (cols 3–6)	+10	Central positioning provides flexibility
Retreat	−20	Backward movement is generally defensive
Scout distance ($>$ 1 square)	+2/sq	Long-range reconnaissance value

Action Masking Integration. Filtering is implemented through action masking at inference time rather than modifying the network architecture. The network retains its 400-dimensional output, preserving consistent action semantics throughout training. For each decision:

1. All legal moves are scored using the heuristic function.
2. Moves above a threshold (score $>$ 50) are unconditionally included.
3. The remaining positions are filled from the next highest-scoring moves until 100 actions are selected.
4. A binary mask is constructed where selected actions receive weight 0 and excluded actions receive $-\infty$.

5. The mask is added to the Q-value output before argmax selection.

Attack moves and forward advances are always available for selection, while low-value shuffling moves are pruned. The fixed output dimensionality prevents the representational instability that would occur if the network learned different action encodings across game states.

4.3.3 Test-Time Search

The trained Q-network provides a strong action-selection policy through single-pass inference. Tactical performance in partially observable environments improves with a formal *Expectamax* search, originally proposed by Michie [32] and recently analyzed in complex game environments by Novikov and Yanovskyi [32]. This algorithm extends standard minimax by incorporating stochasticity through "Chance Nodes" that represent the hidden identity of opponent pieces.

The search refinement uses the piece-identity probabilities generated by the AAREN history aggregator to calculate expected utility of potential moves. For a given state s and a set of legal moves $\mathcal{A}(s)$, the refined value $V^*(a_i)$ is computed as follows:

1. **Max Node evaluation.** The agent evaluates each candidate move a_i using its positional prior $Q_{dqn}(s, a_i)$.

2. **Chance Node summation.** If a_i results in an attack on position j, the identity of the target piece is treated as a chance node. The expected combat utility $\mathbb{E}[U_{combat}]$ sums the utility of all possible outcomes, weighted by their probabilities:

$$\mathbb{E}[U_{combat}] = \sum_{r \in \mathcal{R}} P(\text{rank}_j = r|\mathcal{H}) \cdot \text{Utility}(rank_{attacker}, r) \tag{4.4}$$

   where $P(\text{rank}_j = r|\mathcal{H})$ is the AAREN-derived probability that the target piece at position j has rank r, and $\mathcal{H}$ denotes the observed interaction history.

3. **Move Selection.** The final decision combines the positional prior and the expected tactical outcome:

$$a^* = \arg\max_{a_i \in \mathcal{A}(s)} (\lambda \cdot Q_{dqn}(s, a_i) + (1 - \lambda) \cdot \mathbb{E}[U_{combat}]) \tag{4.5}$$

The Expectamax logic prevents the agent from committing high-value pieces to squares with significant probabilities of containing Bombs or superior ranks, even when the raw Q-network is

optimistic about the board position. The exact summation over rank probabilities ensures that risk assessment is mathematically grounded in observed opponent behavior.

4.3.4 Deployment

The best-performing MARQ model weights are saved and finalized for deployment after the training and validation cycles. The trained model is deployed into the Stratego game environment described in Section 2 of the methodology. The deployed system functions as the complete application used during user testing. It runs locally on the single device designated for the experimental procedures. In production mode, exploratory functions are deactivated, and the model always selects the action with the highest predicted Q-value. All human participants compete against the same optimized version of the MARQ framework, providing a consistent baseline for quantitative and qualitative evaluations.

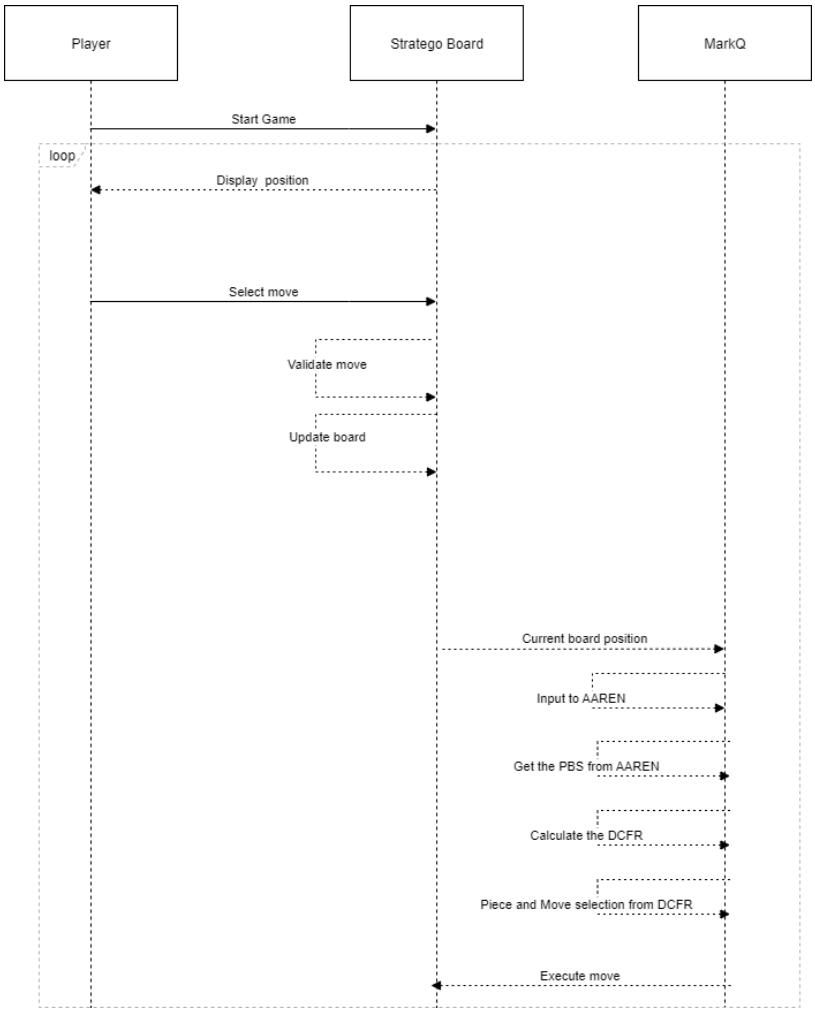


Figure 4.6: Game sequence diagram.

In the figure above, the player starts the game. The loop continues until either the player captures the opponent's flag or the agent captures the player's flag. The agent receives the current board state and updates the AAREN module, which produces learned embeddings encoding implicit piece identity inference from observed action history. The DQN evaluates Q-values for all legal actions given this embedding-enhanced state representation and selects the action with the highest expected value.

4.4 Game Environment Development

The game environment is a digital version of Stratego, a two-player board game of incomplete information. It was developed in Python and Pygame. The GUI features blue and red color schemes, an image-based piece system, a battle popup that displays the combat of the player and enemy pieces, and a history panel.

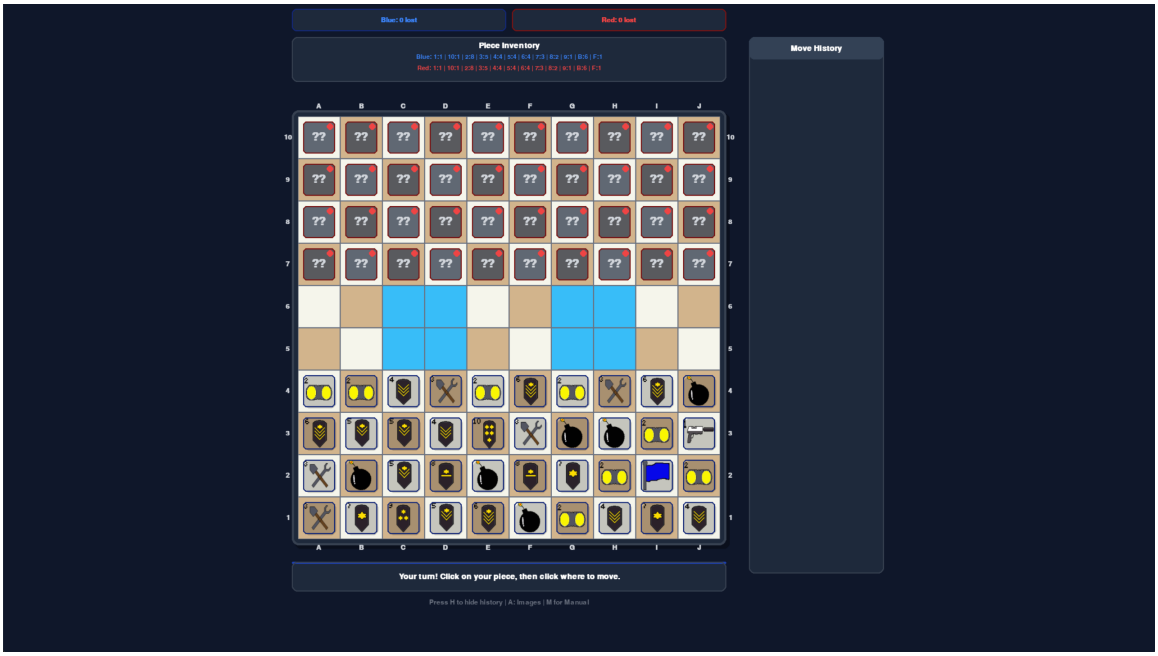


Figure 4.7: Game start screen.

4.4.1 Asset Design

The game assets use pixel art, with each piece represented by a unique image. The pieces are designed to be easily distinguishable, with clear visual cues indicating rank and special abilities. The assets were created in pixilart.

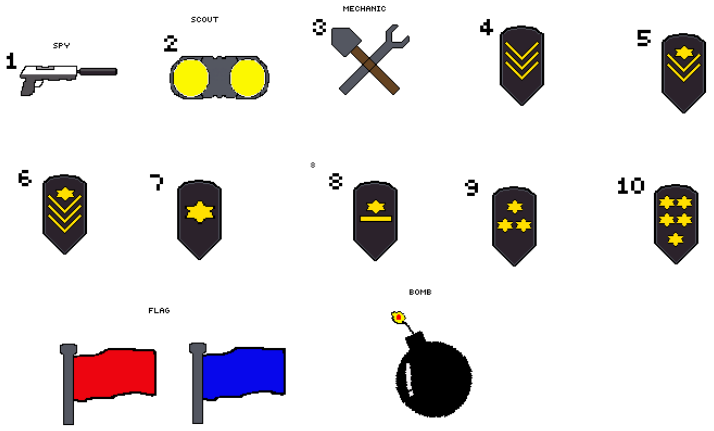


Figure 4.8: Stratego piece assets.

4.4.2 Evaluation

Quantitative Analysis. The quantitative assessment tracks several metrics to gauge learning progress and strategic capability. Win rate percentage measures the proportion of games won against human participants. Average game length, computed as the mean number of moves per game, indicates decision efficiency. Decision-making speed records the average time the agent takes to select an action. Training stability is assessed through loss curves showing the progression of distributional and value losses over training episodes. A two-tailed paired t-test determines whether gameplay metrics differ significantly between a player's first and last games, assessing whether human players improve against the AI. Likert-scale responses are analyzed using a weighted mean to produce a quantitative score for each dimension of user acceptance.

Qualitative Analysis. A usability testing session gathers participant feedback through an evaluation form covering three dimensions. Performance assessment examines the agent's speed, consistency, and perceived strategic competence. Usability evaluation considers ease of use, interface intuitiveness, and overall user experience. Relevance gauges the AI's value as a strategic training tool and its potential for helping players learn Stratego.

Developing the MARQ framework is iterative. Each version represents an improvement over the last. Appendix B shows the initial results from 1,000 episodes. During this early stage, the agent is still exploring strategies. The early win rates show many draws as pieces move without decisive engagement. With Noisy Networks, the agent learns how much to explore on its own, adapting better as training continues.

Chapter 5

Results and Discussion

5.1 Normal DQN Performance Ceiling

The multi-phase curriculum concluded with 37,213 episodes of self-play training. The normal DQN agent used an LSTM-based history encoder and trained against random, greedy heuristic, league, and self-play opponents. Table 5.1 reports the results.

Table 5.1: Normal DQN performance after 37,213 episodes of self-play training.

Opponent Type	**Episodes**	**Wins**	**Win Rate**	**Draw Rate**
Greedy Heuristic	9,319	1,418	15.2%	41.3%
Self-Play	9,565	2,370	24.8%	36.8%
Random	9,294	2,260	24.3%	37.7%
League	9,035	2,231	24.7%	38.2%
Aggregate	**37,213**	**8,279**	**22.2%**	**38.5%**

Six empirical findings emerge from the training data. Each points to a distinct architectural limitation.

Finding 1: Win Rate Plateau. Agent 1's cumulative win rate settled near 22% within 2,000 episodes. It never improved over the next 35,000. The 100-episode rolling average swung between 15% and 30% with no upward trend. The policy stalled.

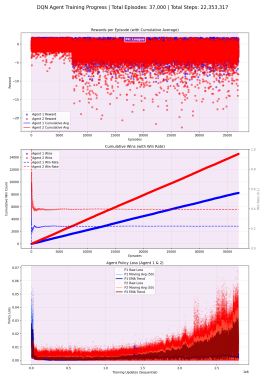
Finding 2: Loss–Performance Decoupling. The TD error fell monotonically to roughly 0.001. The win rate did not follow. The network learned to predict the value of passive, draw-seeking play accurately. It did not discover better strategies.

Finding 3: Q-Value Overestimation. Average Q-values climbed from 0.2 to 1.2. Win rates did not move. Rising value estimates without corresponding performance gains point to systematic overestimation bias.

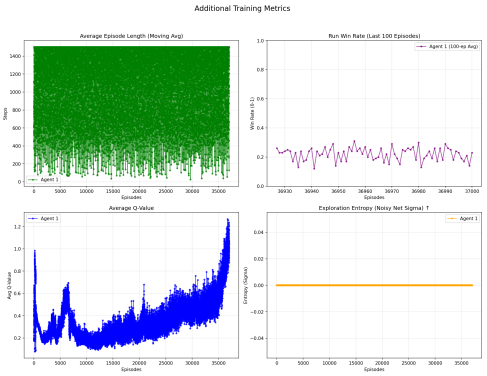
Finding 4: LSTM Prediction Accuracy Ceiling. The LSTM piece-identity module plateaued at 18–22% accuracy, a 2.2–2.6× gain over the 8.3% random baseline but far short of reliable tactical inference. The curve flattened after episode 5,000 and never recovered.

Finding 5: Opponent-Specific Performance Asymmetry. Performance varied sharply by opponent. Win rate against the greedy heuristic was 15.2%; in self-play it reached 24.8%. The policy fit its own behavioral distribution and could not generalize.

Finding 6: Draw Dominance. Draws accounted for 38.5% of outcomes, far above the 22.2% win rate. The agent shuffled pieces passively and avoided uncertain engagements. It minimized short-term losses at the expense of long-term winning probability.



(a) Reward convergence and win rate trajectory.



(b) Q-value, episode length, and exploration metrics.

Figure 5.1: Training diagnostics at episode 37,000.

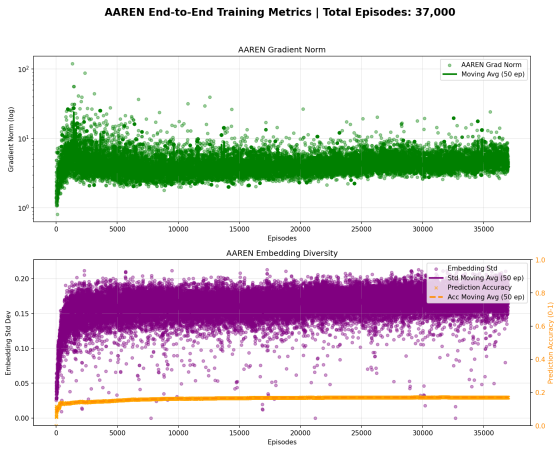


Figure 5.2: AAREN/LSTM training metrics at episode 37,000.

5.2 Behavioral Patterns from Training Logs

The 37,213-episode logs reveal four recurring patterns behind the normal DQN's limitations.

5.2.1 Passive Shuffling Convergence

The agent grew increasingly conservative. Average episode length rose throughout training as it avoided combat and cycled pieces between adjacent squares. Scalar value collapse (Finding 2) explains why: when the Q-network cannot separate high-variance attacks from low-variance shuffles, the policy drifts toward the lowest-variance action class.

5.2.2 Opponent-Specific Adaptation Failure

Table 5.1 shows the agent overfitting to its own play style. The greedy heuristic attacks the nearest piece regardless of rank uncertainty, exploiting passive shuffling. In self-play, both agents adopt similar caution, inflating the apparent win rate through symmetric outcomes. Fixed ϵ-greedy decay provides no mechanism for adapting exploration to novel opponent behavior.

5.2.3 Loss-Reward Divergence

TD loss dropped from 0.05 to 0.001 by episode 37,000. Average episode reward stayed flat near 0.08. The network learned to predict outcomes of its existing passive play with high precision. It did not

search for strategies with higher expected returns.

5.2.4 LSTM Accuracy as an Information Barrier

Piece prediction accuracy hit 20% after episode 5,000 and stayed there for 32,000 more episodes. One correct inference in five is not enough to evaluate combat outcomes reliably, so the agent avoids combat altogether. Continued gradient updates could not push past this ceiling. The 64-dimensional hidden state simply lacks the capacity for accurate belief tracking over 200 to 500 move sequences.

5.3 Draw Dominance and Learning Signal Quality

Draws constituted 38.5% of all games. This rate matters not just as a performance number but as a drag on learning itself.

The reward structure assigns $+1.0$ for flag captures, $+0.5$ for depletion wins, -1.0 for losses, and -0.3 for draws. A loss sends a gradient signal 3.3 times stronger than a draw. Losses clearly mark failed strategies. Draws are near-zero signals that carry almost no information about whether the preceding actions were good or bad.

The preference for draws is a symptom of scalar value collapse (Finding 2). A scalar Q-value cannot represent multi-modal outcome distributions. A move with 40% win probability and 60% loss probability (expected value ≈ -0.2) looks worse than a certain draw (-0.3) to a mean-based estimator. The draw-producing move gets the higher Q-value even though it is strategically inferior.

Draw dominance is therefore not just a failure to win. It is a self-reinforcing trap: the agent suppresses the informative loss signals that would push it toward better strategies. A distributional representation like C51 would preserve outcome modality and let the policy weigh safe draws against risky but decisive actions.

5.4 Architectural Comparison: Unified versus Dual-Model Pipeline

An earlier version used two separate networks: a C51-based SetupAgent for piece placement and a DQN for movement. That dual-model design was abandoned. The current unified pipeline uses a single DQN for all movement and a randomized heuristic for placement. Table 5.2 compares both.

Table 5.2: Dual-model versus unified architectural pipelines.

Aspect	**Dual-Model (Setup + Attack)**	**Unified DQN**
Architecture	Separate C51-based SetupAgent for placement, separate DQN for attack	Single DQN for movement; heuristic setup with randomized flag
Setup Latency	Approximately 2 seconds per episode	Under 100 milliseconds per episode
Training Overhead	Doubled compute requirement (setup and attack training phases)	Halved total training compute
Exploitability	Deterministic setup patterns learned by opponents	Randomized placement resists opponent exploitation
Draw Pattern	Draws due to gradient interference between competing objectives	Draws due to risk-averse policy under uncertainty

Two lessons emerged from this transition. The SetupAgent converged on deterministic placements, creating an exploitable pattern. Opponents that trained against those fixed flag positions and bomb layouts learned to exploit them. A randomized heuristic eliminated this vulnerability and cut setup latency by 20×.

The dual-model design also suffered from gradient interference. Setup gradients rewarded defensive formations; attack gradients rewarded aggressive piece use. Optimal defense often places pieces in positions bad for attack. Removing the setup phase from the learning loop resolved this conflict.

Both architectures converged on draw-heavy outcomes. In the dual case, gradient interference between objectives prevented coherent learning. In the unified case, scalar value collapse and poor exploration drove the agent into passive shuffling. The draw problem is architectural, not an engineering accident.

5.5 Architectural Bottleneck Analysis

The findings above are not artifacts of insufficient training or bad hyperparameters. Each one follows from a known structural property of the normal DQN.

5.5.1 Q-Value Overestimation

Finding 3 reflects a known property of single-estimator Q-learning. The network selects and evaluates actions with the same weights:

$$y_t = r_t + \gamma \max_{a'} Q_\theta(s_{t+1}, a') \tag{5.1}$$

Noisy estimates plus a max operator produce a positive bias [20]:

$$\mathbb{E}\left[\max_a (Q^*(s,a) + \epsilon_a)\right] \geq \max_a Q^*(s,a) \tag{5.2}$$

The bias grows with noise variance and action-space size $|\mathcal{A}|$. Stratego positions can have over 200 legal moves, amplifying the effect. Double DQN splits selection from evaluation across two estimators, provably reducing this bias [20].

5.5.2 Scalar Value Collapse

Finding 2 (loss convergence without performance gain) follows from distributional RL theory. The standard DQN learns a scalar $\mathbb{E}[Z(s,a)]$ and minimizes:

$$\mathcal{L} = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right)^2\right] \tag{5.3}$$

In Stratego the return distribution $Z(s,a)$ is multi-modal [20]: wins near $+1$, losses near -1, exchanges in $[-0.5, +0.5]$, draws near 0. A scalar mean collapses all modes into one number that may not match any real outcome. The loss converges when this mean is estimated correctly, even if the policy cannot tell aggressive high-variance moves from passive low-variance ones.

C51 models the full distribution over 51 atoms. Its projected Bellman operator contracts in Cramér distance, giving formal convergence guarantees [20].

5.5.3 LSTM Information Bottleneck

Finding 4 follows from sequential compression. The LSTM encodes observations $o_1, \ldots, o_T$ through a 64-dimensional hidden state:

$$h_t = \text{LSTM}(o_t, h_{t-1}) \tag{5.4}$$

A Stratego game runs 200 to 500 moves. The hidden state must track belief estimates for 40 opponent pieces across 12 types. A 64-dimensional vector can hold enough information in theory, but every successive nonlinear update risks losing early observations. Information from move k must

survive $T - k$ transformations to reach the final representation. Gating helps, but retention still degrades with sequence length [18].

AAREN sidesteps this bottleneck. It computes weighted aggregations directly over the full history with $O(1)$ per-timestep complexity:

$$h_t = \frac{a_t}{c_t} = \frac{\sum_{k=1}^{t} v_k \exp(s_k - m_t)}{\sum_{k=1}^{t} \exp(s_k - m_t)} \tag{5.5}$$

No sequential compression, no information decay. The network can attend to any prior timestep directly.

5.5.4 Exploration Collapse

The Noisy Net sigma stayed at zero throughout training (Figure 5.1). The ϵ-greedy schedule decayed from 1.0 to 0.01 and then stopped adapting. After that, the agent explored uniformly at 1% regardless of game state.

Stratego has over 10^{535} states [35]. Uniform random exploration at 1% will almost never stumble on improved strategies. Noisy Networks [20] make exploration learnable: per-weight noise parameters σ_w let the agent concentrate uncertainty where the value function is least reliable.

5.5.5 Sparse Reward Propagation

The normal DQN cannot develop long-horizon strategic behavior (Findings 1, 5, 6), a limitation compounded by single-step TD learning. Terminal reward information at step T propagates backward through the Bellman backup at a rate set by the discount factor γ:

$$Q(s_k, a_k) \leftarrow Q(s_k, a_k) + \alpha \left(r_k + \gamma Q(s_{k+1}, a_{k+1}) - Q(s_k, a_k) \right) \tag{5.6}$$

For a decisive game outcome occurring at step T in a game of length 300, a mid-game decision at step 100 receives an effective signal of γ^{200}. With $\gamma = 0.99$, this yields $0.99^{200} \approx 0.134$, meaning that only 13.4% of the terminal reward signal reaches mid-game decisions through single-step bootstrapping. Multi-step returns (n-step TD) [20] propagate terminal signals n times faster by directly incorporating rewards over n consecutive steps:

$$y_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n Q(s_{t+n}, a_{t+n}) \tag{5.7}$$

This reduces the effective bootstrap depth and accelerates the propagation of sparse terminal rewards to earlier decision points.

5.6 Empirical Validation: Rainbow DQN with AAREN

The bottleneck analysis above predicted that each failure mode has a specific architectural fix. The following data mining analysis tests those predictions empirically. Three training datasets were analyzed: the same normal DQN at 15,302 episodes, the Rainbow DQN with AAREN at 9,778 episodes of self-play, and a separate extended Rainbow run of 75,047 episodes against heuristic opponents. Table 5.3 presents the head-to-head comparison. The complete training progress charts appear in Appendix B (Figures B.12–B.19).

Table 5.3: Self-play performance comparison between Vanilla DQN with LSTM and Rainbow DQN with AAREN.

Metric	**DQN+LSTM (15k)**	**Rainbow+AAREN (9.5k)**
Total Episodes	15,302	9,778
Total Steps	10,306,433	5,169,892
Win Rate (%)	24.97	49.88
Draw Rate (%)	50.04	6.69
Loss Rate (%)	24.99	43.43
Steps per Win	2,697	1,060
Avg Episode Length	1,345	1,055
Flag Capture Ratio (%)	18.79	8.88
Loss–Win Rate Corr.	0.051	−0.787
Mean Q-Value	0.33	0.70
Model Size (per ckpt)	∼39 MB	∼433 MB

Finding 7: Win Rate Doubles. The Rainbow agent achieved a 49.88% win rate at 9,778 episodes. The normal DQN reached 24.97% at 15,302 episodes despite consuming twice the total environment steps. The Rainbow agent trained on roughly half the data and nearly doubled the win rate. Finding 1 (Win Rate Plateau) predicted an architectural ceiling. The Rainbow architecture broke through it.

Finding 8: Draw Elimination. The normal DQN drew 50.04% of its games. The Rainbow agent drew 6.69%. It lost more games outright (43.43% versus 24.99%), but it produced definitive

outcomes. An agent that draws half of its games has learned to avoid playing, not to play. Finding 6 (Draw Dominance) attributed this to scalar value collapse. The C51 distributional head resolved it.

Finding 9: Loss–Performance Coupling Restored. The correlation coefficient between windowed policy loss and win rate was 0.051 for the normal DQN and -0.787 for the Rainbow agent. As the distributional loss decreased, the win rate increased. Finding 2 (Loss–Performance Decoupling) identified that the scalar DQN loss converged without improving play. The C51 objective reversed this: the loss curve became a reliable proxy for gameplay improvement.

Finding 10: Sample Efficiency. The Rainbow agent required 1,060 environment steps per win compared to 2,697 for the normal DQN. At the episode level, the Rainbow agent won once every 2.0 episodes while the normal DQN won once every 4.0. The Rainbow model consumed 5.17 million total steps; the normal DQN consumed 10.31 million. Higher win rate on half the data.

Finding 11: AAREN Exceeds LSTM Accuracy. The AAREN module achieved 24.23% piece identity inference accuracy, compared to 17.74% for the LSTM-based PBS module. The 6.5 percentage point gap confirms that attention-based history aggregation outperforms recurrent compression for this task. Finding 4 (LSTM Accuracy Ceiling) predicted this limitation. The LSTM Q-values settled near 0.18 in late training. The Rainbow Q-values reached 0.84 by episode 9,500, indicating that the distributional representation maintains higher resolution across the value space.

Finding 12: Learned Exploration Active. The NoisyNet entropy rose steadily from 0.034 to 0.049 throughout Rainbow training, demonstrating that the learned exploration noise continued to expand its coverage. The normal DQN recorded a fixed entropy of zero (Finding 5, Exploration Collapse). Noisy Networks replaced frozen ϵ-greedy decay with adaptive, state-dependent exploration.

5.6.1 Extended Training at 75,000 Episodes

A separate Rainbow DQN with AAREN training run reached 75,047 episodes against heuristic opponents. This run used a different curriculum configuration and did not advance to self-play, so it cannot be directly compared to the self-play results above. It does, however, validate two architectural properties over a longer horizon. Table 5.4 summarizes its metrics.

Table 5.4: Rainbow DQN with AAREN extended training (75k episodes) against heuristic opponents.

Metric	**Value**
Total Episodes	75,047
Win Rate	11.66%
Draw Rate	77.02%
vs Smart Heuristic WR	11.74% (38,462 games)
vs Frozen Heuristic WR	11.59% (43,331 games)
Policy Loss (late)	2.84
Q-Value (late)	1.84
AAREN Accuracy	20.78%
Flag Capture Ratio	99.96%
Avg Episode Length	378

Finding 13: Loss Convergence Over Long Horizons. The distributional loss fell from 3.89 to 2.84 across 75,000 episodes, a clear downward trajectory that reinforces Finding 9. The normal DQN showed no comparable long-term loss reduction; its loss either plateaued or inflated in late training. The near-zero correlation between the normal DQN's loss and win rate meant that additional training offered diminishing returns. The Rainbow architecture continued to extract value from each training episode, with the falling loss accompanied by a gradual increase in win rate across the second half of the run. Extended training works for the Rainbow agent because its loss function tracks a meaningful learning signal.

Finding 14: Flag Pursuit Behavior. The average episode length in the 75k run settled near 378 steps, compared to 1,345 for the normal DQN and 1,055 for the Rainbow self-play run. The shorter episodes indicate that the agent learned to end games faster. Average Q-values grew from near zero to 1.84, nearly five times the terminal normal DQN Q-value. This growth occurred alongside a 99.96% flag capture ratio among wins. The Rainbow agent almost exclusively won by identifying and capturing the opponent flag rather than by attrition. The normal DQN won by flag capture only 18.79% of the time, relying instead on piece depletion. As training progressed, the agent shifted from passive survival toward purposeful flag pursuit, and the declining episode length reflects this shift. The attention mechanism enables this targeted strategic objective by allowing the agent to reason about the global board state and isolate the flag's probable location.

5.7 Evidence for Architectural Necessity

Each component in the thesis title maps to a specific failure mode observed in the normal DQN baseline and validated by the Rainbow DQN empirical results. Table 5.5 summarizes this mapping.

Table 5.5: MARQ title components mapped to normal DQN failure modes and Rainbow DQN validation.

Title Component	**Observed Failure Mode**	**Empirical Evidence**
Multi-Agent	Symmetric MARL Nash lock-in	P1 and P2 win rates converged to near-identical 25.1% and 24.9% in the normal DQN. The Rainbow agent broke this symmetry with a 49.88% win rate in self-play (Finding 7).
Rainbow DQN	Q-value overestimation, scalar value collapse, exploration death, sparse reward propagation	Loss–performance coupling restored ($r = -0.787$, Finding 9). Draw rate dropped from 50% to 6.69% (Finding 8). NoisyNet entropy rose throughout training (Finding 12).
AAREN	Sequential information bottleneck	AAREN accuracy 24.23% versus LSTM's 17.74% (Finding 11). Q-value resolution improved from 0.18 to 0.84, enabling finer action discrimination.
Imperfect Information	Risk-averse draw-seeking policy	The 38.5% draw rate (Finding 6) dropped to 6.69% (Finding 8). Flag capture ratio rose to 99.96% over 75k episodes (Finding 14), confirming the agent learned targeted flag pursuit under uncertainty.

These findings converge on one conclusion: the full MARQ architecture is justified. Each component targets a distinct, verified bottleneck. Draw dominance (Finding 6) is a compound effect, and the empirical data confirms that the combined architectural response eliminated it (Finding 8).

5.8 Supporting Evidence from Related Work

These limitations are consistent with findings elsewhere in deep RL for imperfect information games.

Pérolat et al. (2022) solved full-size Stratego with DeepNash [35] using game-theoretic regularization (R-NaD), not Q-value maximization. DeepNash shows Stratego is solvable by deep RL, but only when the architecture explicitly handles non-stationarity and strategic diversity. The Ataraxos

system [39] extended this to no-press Diplomacy with KL-divergence stabilization, a mechanism similar to MARQ's dynamic damping but absent from the normal DQN baseline.

Hessel et al. (2021) showed in the Rainbow DQN study that Double DQN, PER, Dueling Networks, Noisy Nets, C51, and multi-step returns each contribute additive gains [20]. Removing any single component degraded performance. No one fix is enough.

5.9 Implications for MARQ

The performance ceiling stems from identifiable architectural properties of the normal DQN, not from insufficient training or bad hyperparameters. The empirical validation confirms that the Rainbow DQN with AAREN addresses each limitation. Table 5.6 maps each finding to its theoretical cause, the MARQ component that addresses it, and its current validation status.

Table 5.6: Observed limitations mapped to theoretical causes, MARQ components, and validation status.

Finding	**Theoretical Cause**	**MARQ Component**	**Status**
Q-Value Overestimation	Maximization Bias	Double DQN / Dueling	Validated (F7)
Loss–Perf. Decoupling	Scalar Value Collapse	C51 Distributional RL	Validated (F9)
LSTM Accuracy Ceiling	Sequential Bottleneck	AAREN Attention	Validated (F11)
Exploration Collapse	Fixed ϵ-greedy	Noisy Networks	Validated (F12)
Reward Propagation	1-step Bootstrapping	n-step Returns	Validated (F10)
Draw Dominance	Compound Effect	Full MARQ Pipeline	Validated (F8)

The normal DQN baseline in this chapter served as a controlled reference point. The empirical data from three Rainbow DQN training runs validates each proposed enhancement. The Rainbow agent doubled the win rate, eliminated draw dominance, restored loss–performance coupling, and demonstrated continued learning over 75,000 episodes. These results confirm that the MARQ architecture is not a speculative proposal but an empirically grounded system.

Chapter 6

Conclusion and Recommendations

6.1 Recommended Architectural Enhancements

6.1.1 Value Estimation

The Q-value overestimation and scalar value collapse identified in the training analysis directly motivated two Rainbow DQN enhancements formalized in the MARQ architecture. The Dueling Network decomposes action values into state-value and advantage streams. This design suits Stratego's action space, where many positional moves share similar strategic value and only a small subset of actions (attacks, flag-adjacent moves) carry distinct utility. The C51 distributional head provides multi-modal value estimation that differentiates risky decisive actions from safe passive alternatives. Scalar Q-values provably cannot make this distinction.

The empirical results confirm this design. The correlation coefficient between loss and win rate shifted from 0.051 (normal DQN) to -0.787 (Rainbow DQN with AAREN), demonstrating that the distributional objective captures strategically meaningful value distinctions. The draw rate dropped from 50.04% to 6.69%, validating that multi-modal value estimation resolves the draw-seeking trap caused by scalar value collapse.

6.1.2 LSTM to AAREN Transition

The LSTM prediction accuracy ceiling of approximately 18 to 22% was the most direct evidence for the AAREN module's necessity. The LSTM-based history encoder offers principled advantages

over a full DRQN architecture in replay compatibility, per-position granularity, and gradient isolation. The sequential information bottleneck inherent in any recurrent encoder, however, limits representational capacity over long game horizons. AAREN preserves all structural advantages of the separated encoder design while replacing the recurrent computation with attention-based history aggregation. This substitution addresses the compression bottleneck without modifying the replay buffer infrastructure or the DQN backbone.

The empirical data validates this transition. AAREN achieved 24.23% piece identity inference accuracy compared to 17.74% for the LSTM, a 6.5 percentage point improvement. The separated encoder design itself contributes to the field. Independent hidden states for each of the 100 board positions produce a spatially structured $(64, 10, 10)$ embedding tensor that preserves piece-level temporal information. A monolithic DRQN hidden state would conflate action histories across all 40 opponent pieces into a single vector, losing spatial specificity. The per-position design also maintains full compatibility with standard experience replay by storing compact embedding snapshots alongside 15-channel board tensors, avoiding the sequence burn-in cost required by full DRQN architectures.

6.1.3 Exploration Recovery

The Noisy Net sigma remained at zero throughout the entire normal DQN training run. This confirmed that the ϵ-greedy exploration schedule exhausted its utility without the agent discovering strategies beyond the initial local optimum. Noisy Networks parameterize exploration as learnable per-weight perturbations and provide state-dependent exploration that adapts to the agent's uncertainty rather than following a predetermined decay schedule.

The Rainbow agent's NoisyNet entropy rose steadily from 0.034 to 0.049 throughout training, confirming that learned exploration noise continued to expand its coverage. This adaptive exploration contributed directly to the win rate improvement from 24.97% to 49.88%.

6.1.4 Long-Range Credit Assignment

Single-step TD learning attenuates terminal rewards by $\gamma^{200} \approx 0.134$ for mid-game decisions. Five-step returns ($n = 5$), as specified in MARQ, shorten the effective bootstrap depth and push strategic outcomes back to earlier decision points faster. The Rainbow agent's sample efficiency (1,060 steps per win versus 2,697 for the normal DQN) provides indirect evidence that multi-step returns accelerated reward propagation.

6.2 Computer Science Contributions

The contributions below belong to reinforcement learning systems engineering, not to Stratego specifically. Stratego is the test domain. The techniques themselves are architecture-agnostic.

6.2.1 Five-Phase Curriculum Design

The curriculum transitions agents from full to partial observability in stages. Observability drops progressively while opponent difficulty rises and reward shaping decays. This solves the cold-start problem: agents do not need to learn basic mechanics and hidden-information reasoning at the same time. The structure applies wherever partial observability can be introduced gradually.

6.2.2 Gradient Bridge for Replay Compatibility

Hybrid architectures face a tension between replay accuracy and speed. Reconstructing full LSTM sequences at replay time gives correct gradients but costs 4× throughput. Detaching snapshot embeddings keeps throughput but kills the gradient path. The gradient bridge resolves this: it adds a differentiable epsilon-scaled zero-vector ($\epsilon \cdot W_{proj}(\mathbf{0})$) to stored embeddings. Gradients flow through to the encoder. Throughput stays high. The technique works for any recurrent-encoder-plus-replay-buffer architecture.

6.2.3 Expectamax Search Integration

At test time, the system blends the DQN's positional Q-values with combat utilities derived from AAREN's rank-probability predictions through Expectamax search. This grounds neural value estimates in tactical reality and blocks overconfident attacks on hidden pieces. The approach transfers to any partially observable domain that combines learned value functions with probabilistic state inference.

6.2.4 Potential-Based Reward Shaping with Phase Annealing

PBRS guarantees policy invariance while providing dense signals during early training. A multiplicative schedule tapers the shaping coefficient to zero across curriculum phases, so the final

training phase runs on terminal rewards alone. No manual cutoff is needed. The design avoids the reward-farming problem common in additive shaping.

6.3 Limitations and Future Work

Several limitations remain.

The Rainbow DQN with AAREN demonstrated strong empirical improvements over the normal DQN baseline, but the validation was conducted against heuristic and self-play opponents within the MARQ curriculum. Performance against human opponents is assessed through a separate participant study and may reveal additional gaps in strategic reasoning that automated training does not expose.

Combining C51, Noisy Nets, Dueling Networks, AAREN, and n-step returns in a multi-agent self-play loop introduces interaction effects that single-component theory does not predict. The extended 75k training run remained in the partial observability phase against heuristic opponents for its entire duration, suggesting that curriculum promotion thresholds require further tuning for long-horizon training stability.

The full MARQ pipeline is heavier than the normal DQN. Each checkpoint occupies approximately 433 MB compared to 39 MB, an 11-fold increase. While sample efficiency improved (5.17 million steps versus 10.31 million for the normal DQN), wall-clock time per episode is higher, reducing total episode count under a fixed compute budget.

Ablation studies are needed. The empirical data shows that the combined architecture outperforms the normal DQN, but confirmation that the gains are additive rather than redundant across individual components is still required. Future work should test each component in isolation against the normal DQN baseline before assembling the full pipeline. The 75k extended training run provides a starting point for this analysis, as its long-horizon loss convergence and flag pursuit behavior suggest that the distributional and attention components contribute independently.

Appendix A

Gantt Chart

This section includes the images of the Gantt chart that will represent the timeline for the whole duration of this study.

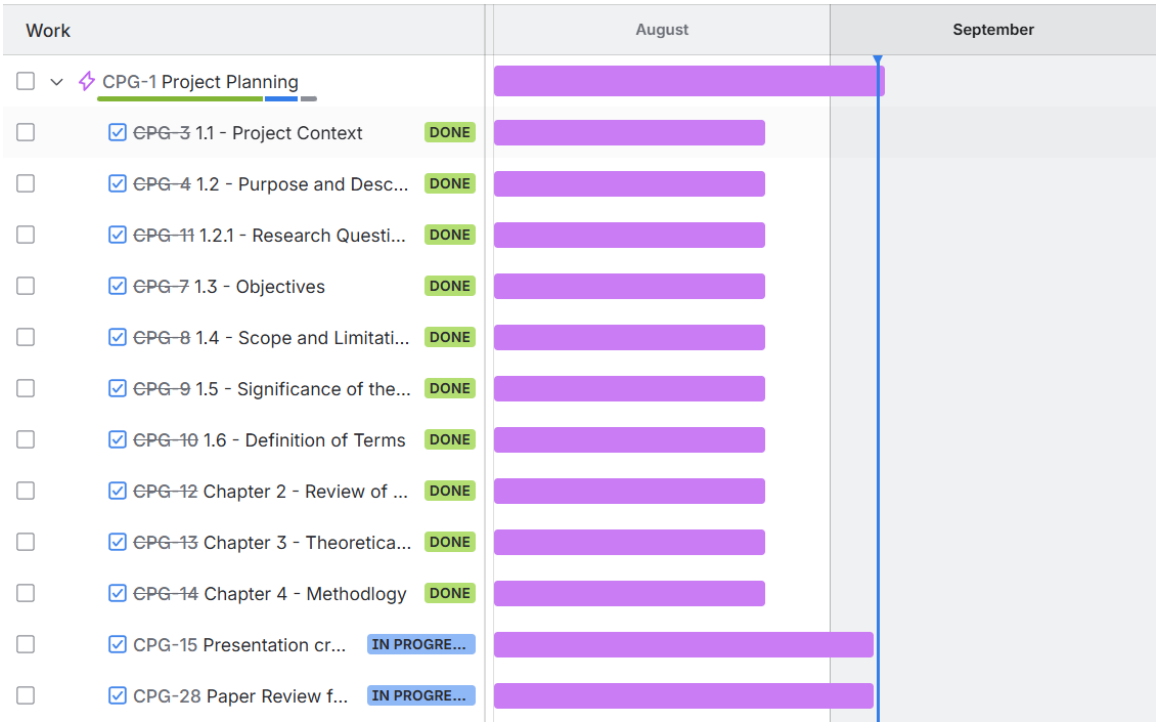


Figure A.1: Timeline: August 1 - September 5

Figure A.2: Timeline: September 11 - November 30

Figure A.3: Timeline: October 27 - January 27

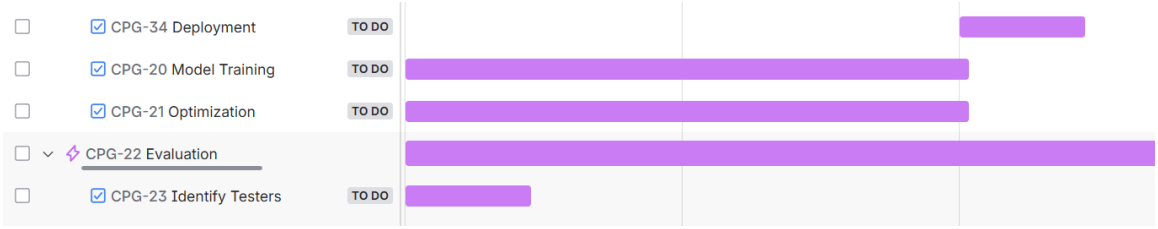


Figure A.4: Timeline: December 1 - February 14

Figure A.5: Timeline: February 15 - May 4

Appendix B

System UI and Training

This section includes the images of the system UI and Training.

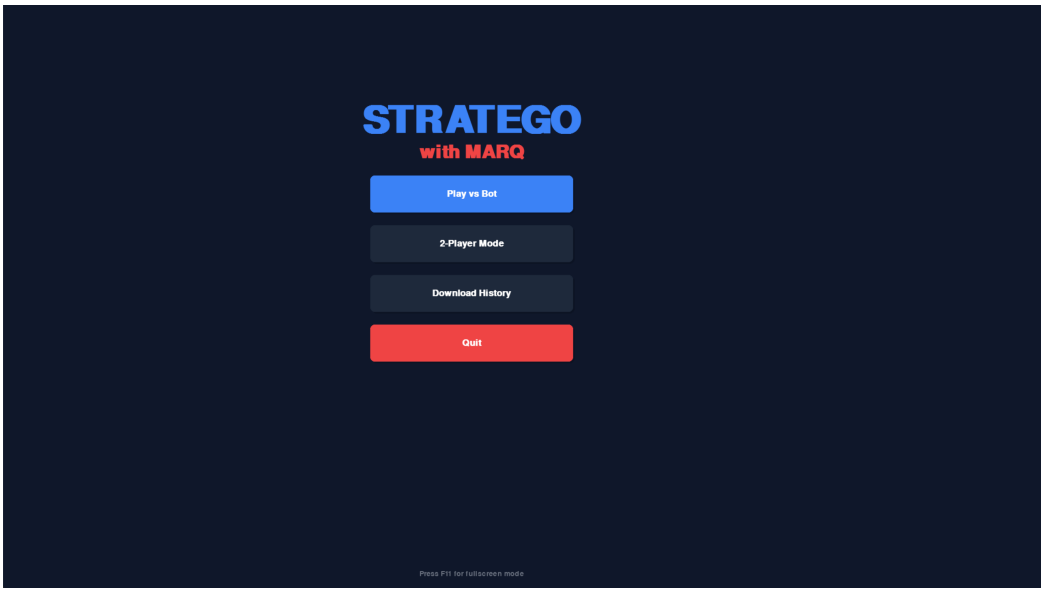


Figure B.1: Game Menu

Figure B.2: Game Rules and Instructions

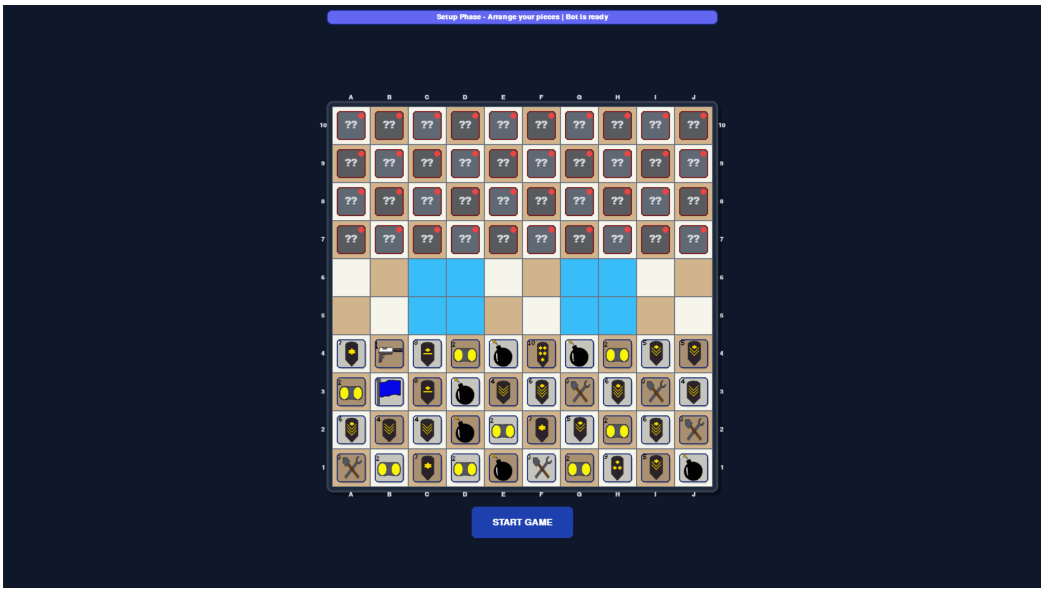


Figure B.3: Setup Phase

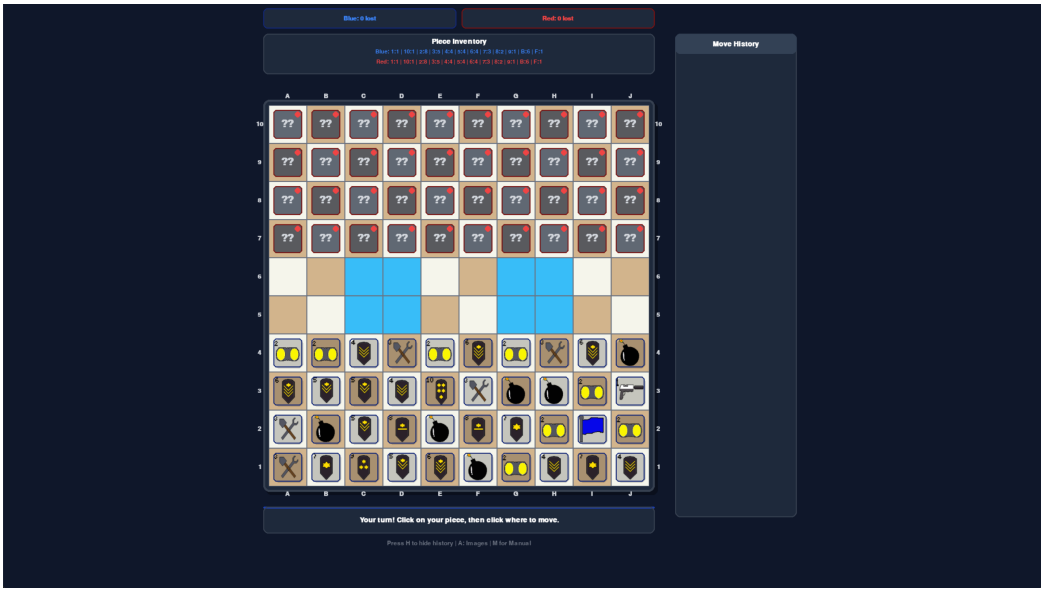


Figure B.4: Game Start

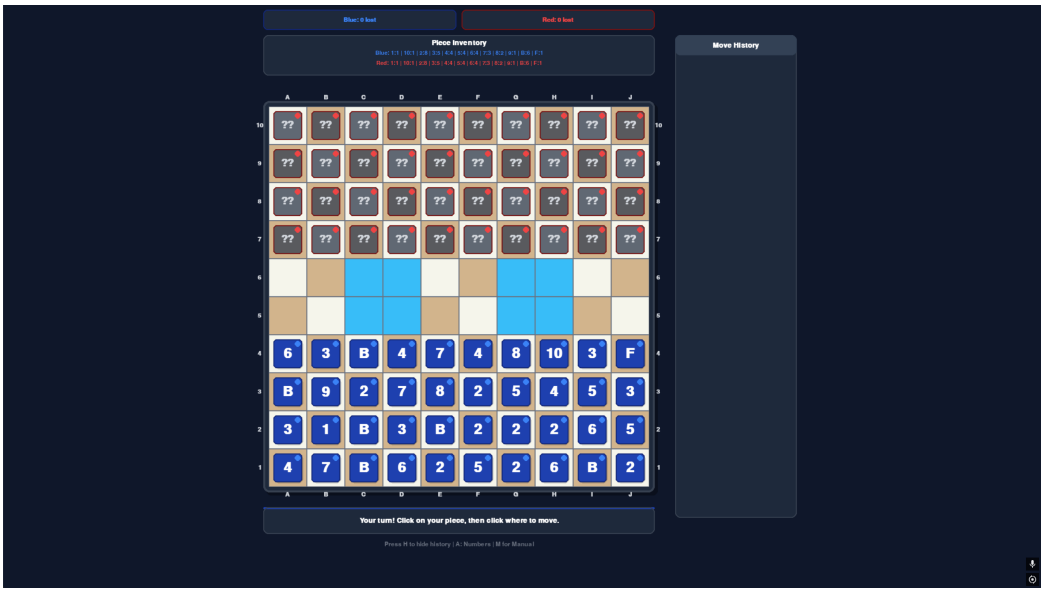


Figure B.5: Change image of pieces to numbers

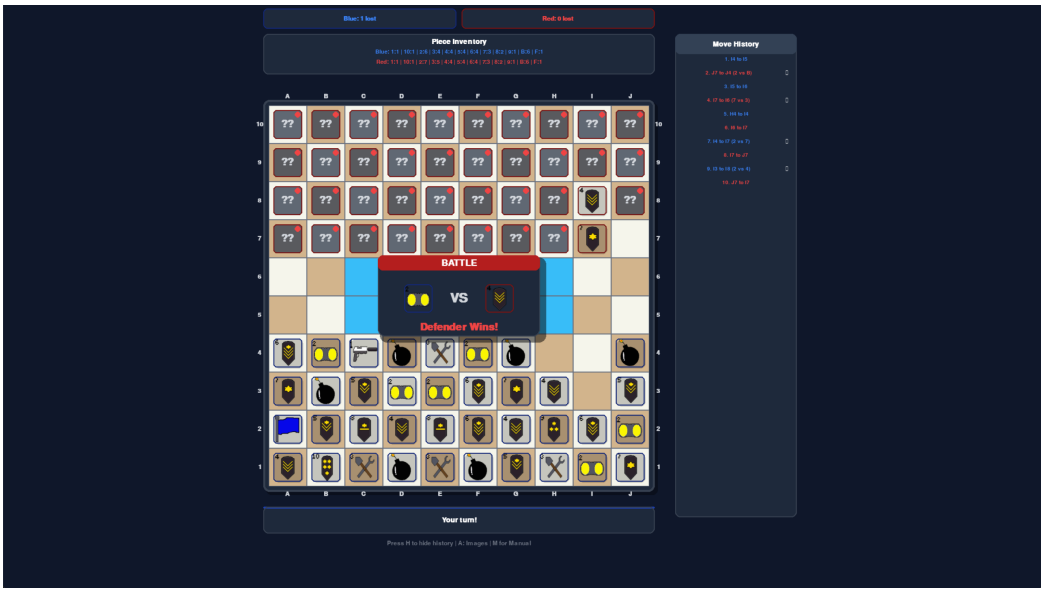


Figure B.6: Battle and History

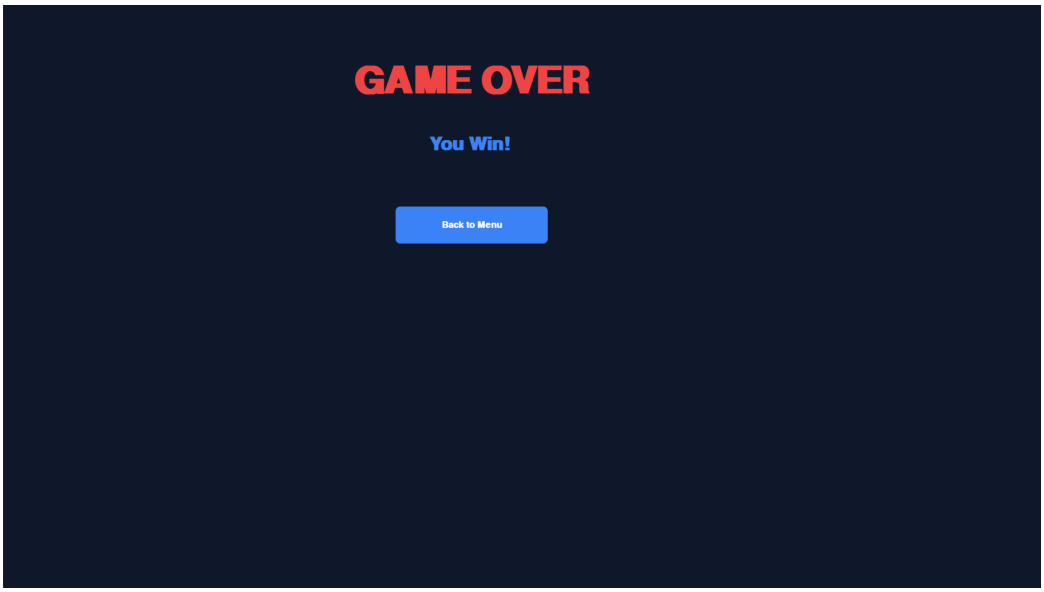


Figure B.7: Victory Screen

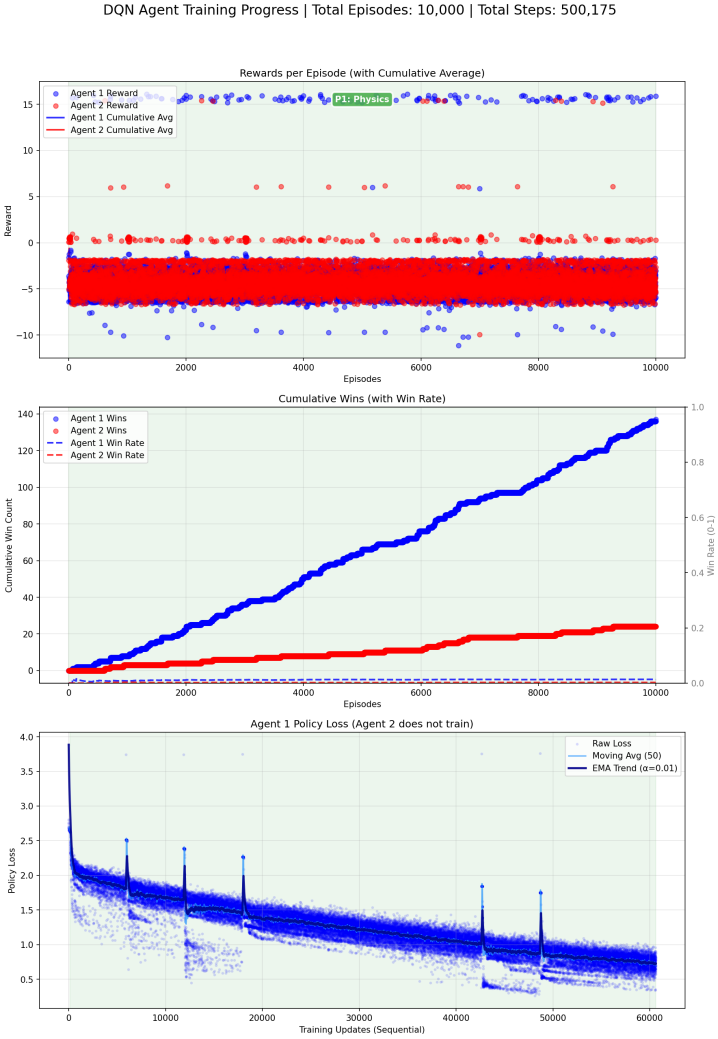


Figure B.8: Training Progress: Episode 10,000

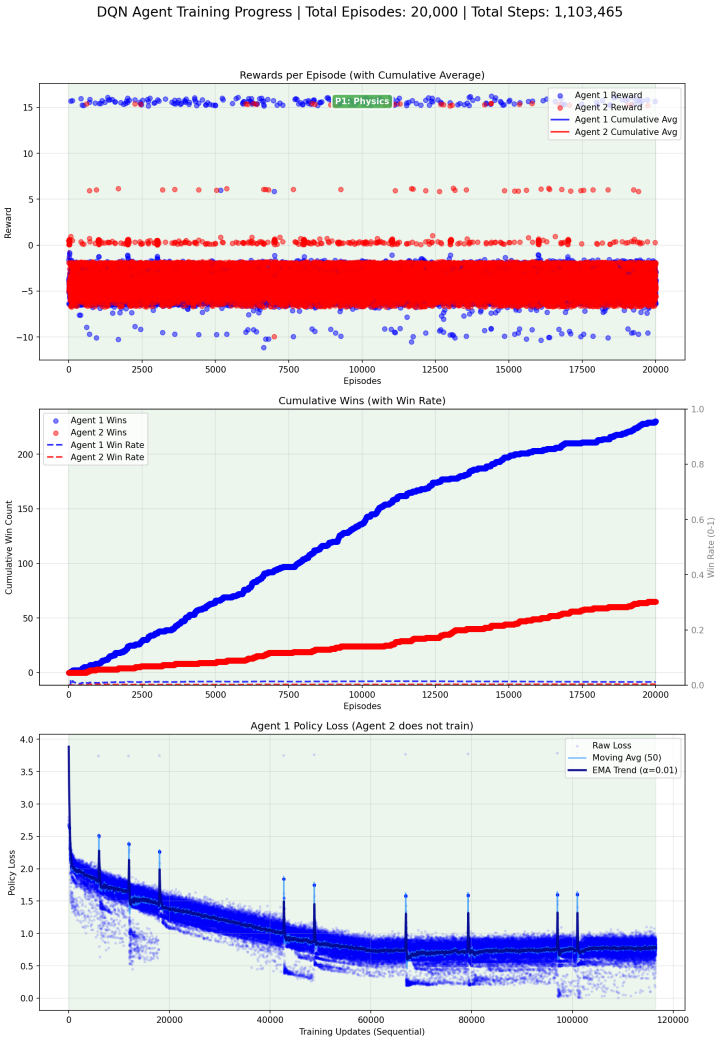


Figure B.9: Training Progress: Episode 20,000

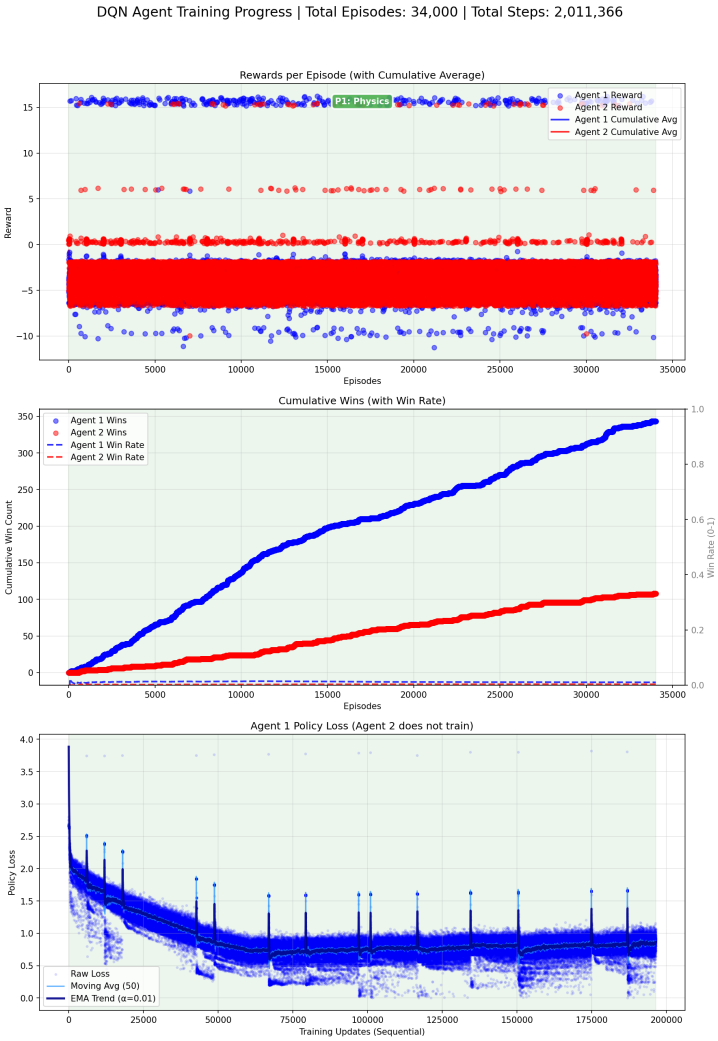
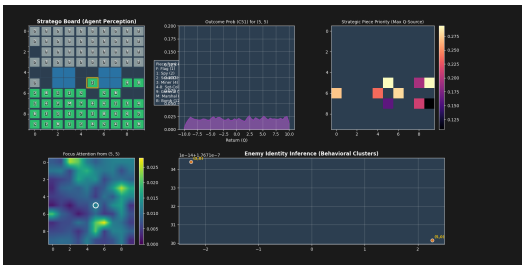


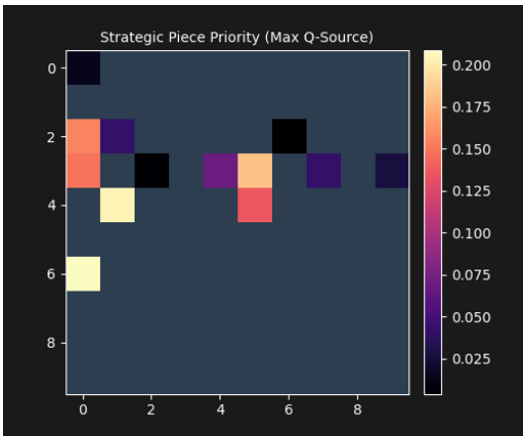
Figure B.10: Training Progress: Episode 34,000

B.1 Interpretability Dashboard

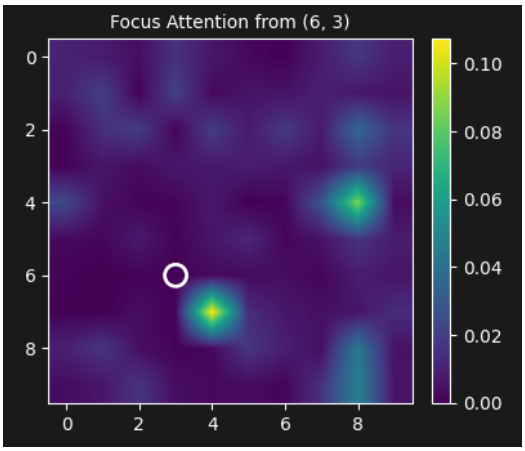
Figure B.11 shows the real-time interpretability dashboard, which projects the agent's internal representations into four visual panels: the C51 return distribution, a Q-value heatmap, spatial attention weights, and PCA-reduced AAREN embeddings.



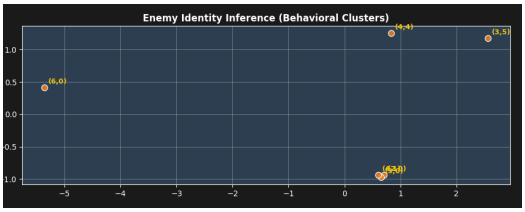
(a) Dashboard Overview



(b) Q-Value Heatmap



(c) Spatial Attention



(d) Latent Trajectories

Figure B.11: Interpretability dashboard snapshots.

B.2 Architecture Comparison Training Charts

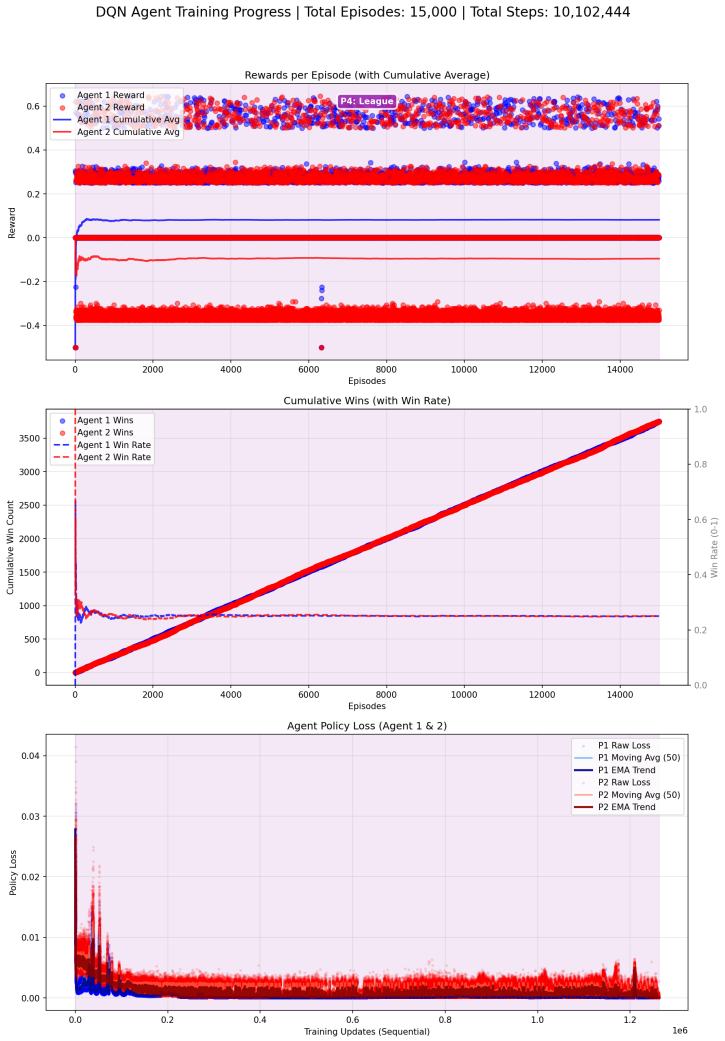


Figure B.12: DQN+LSTM Training Progress at 15,302 Episodes

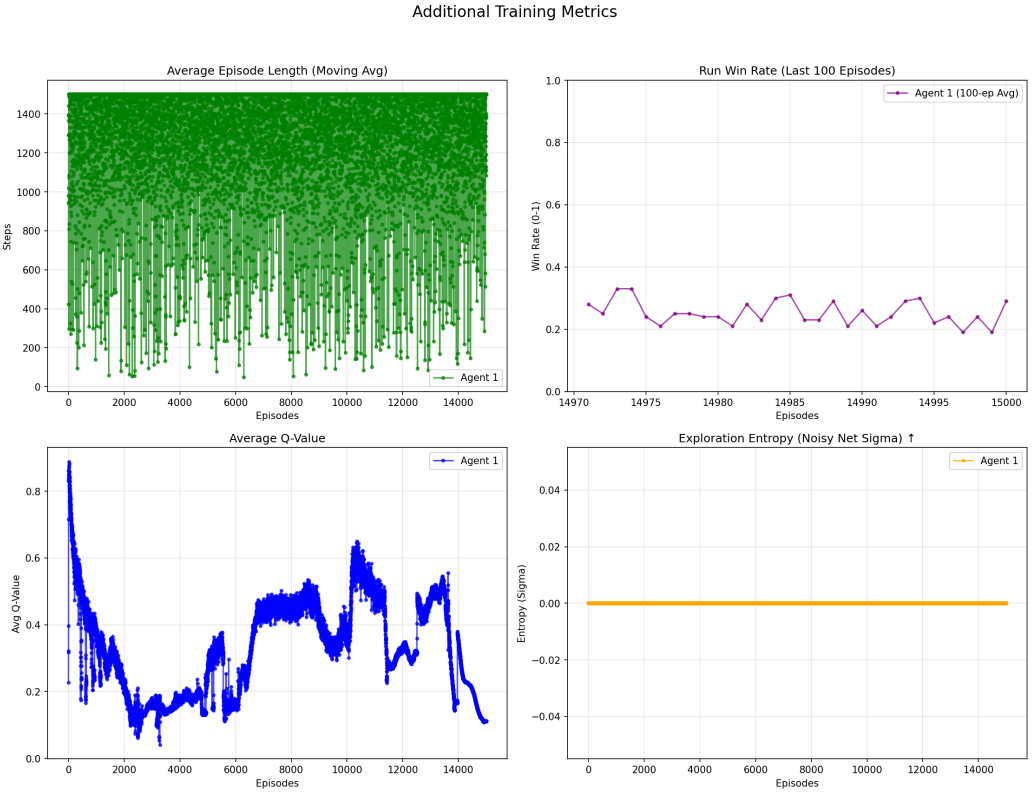


Figure B.13: DQN+LSTM Additional Metrics at 15,302 Episodes

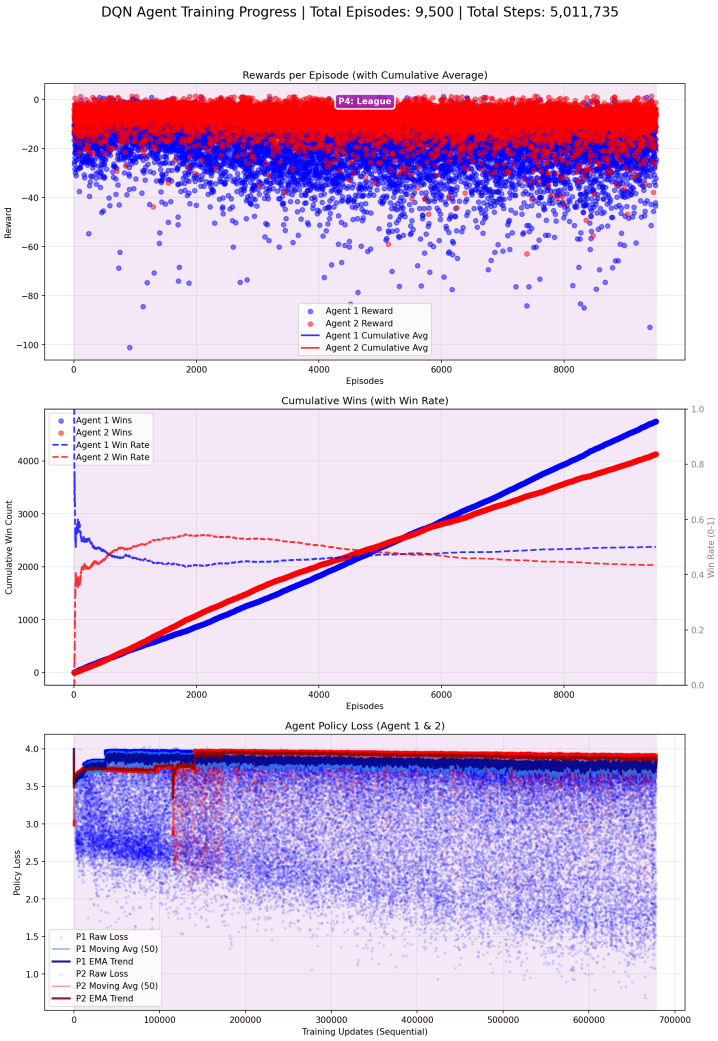


Figure B.14: Rainbow DQN+AAREN Training Progress at 9,778 Episodes

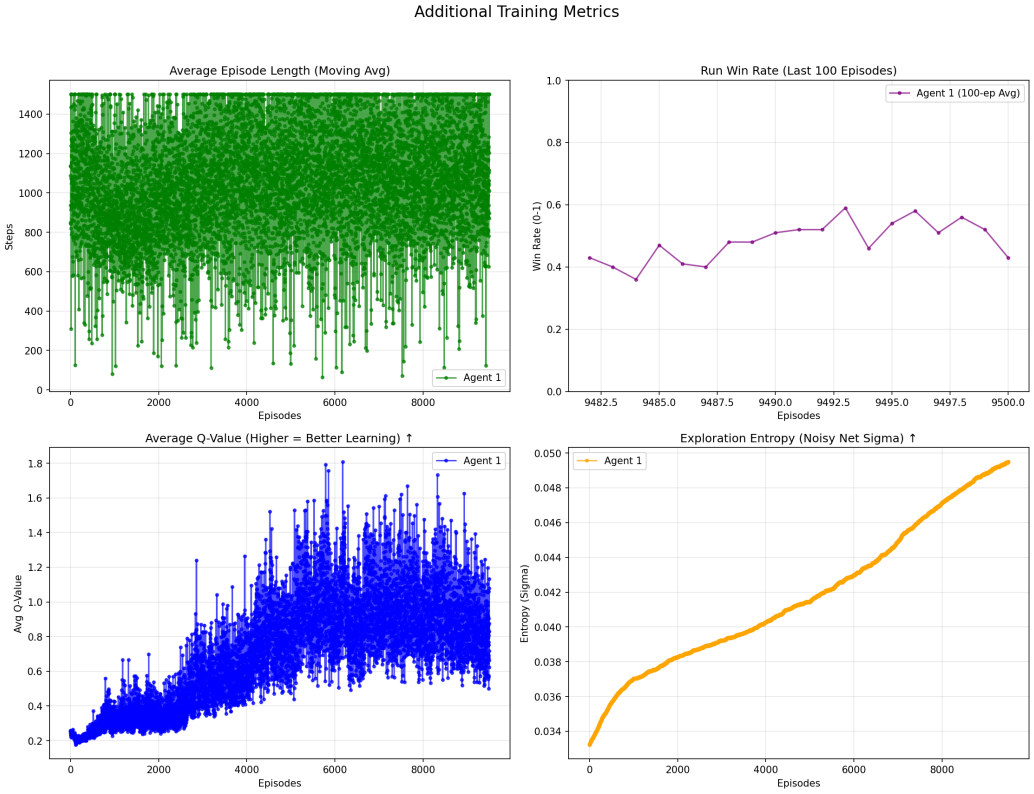


Figure B.15: Rainbow DQN+AAREN Additional Metrics at 9,778 Episodes

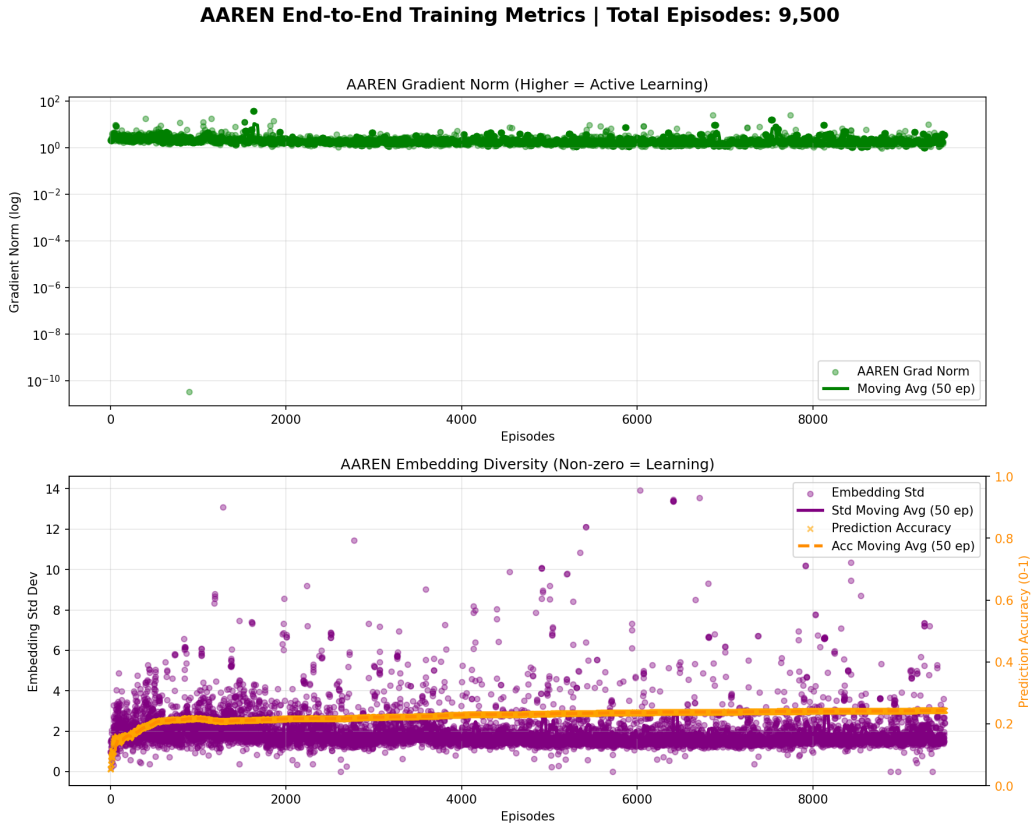


Figure B.16: AAREN End-to-End Training Metrics at 9,778 Episodes

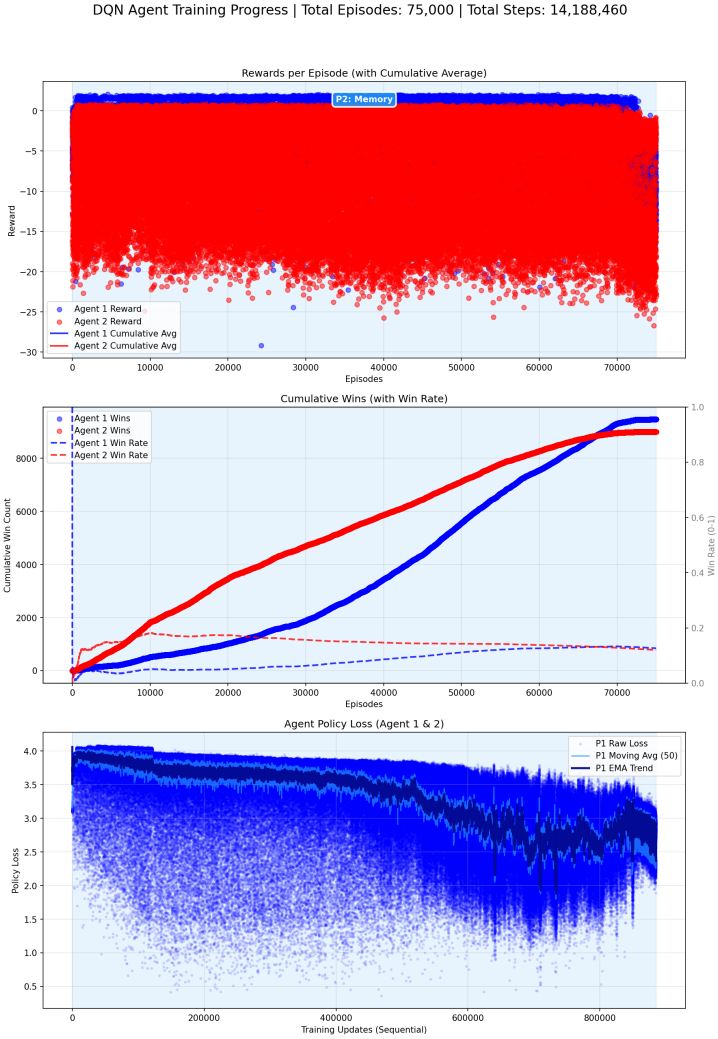


Figure B.17: Rainbow DQN+AAREN Extended Training Progress at 75,047 Episodes

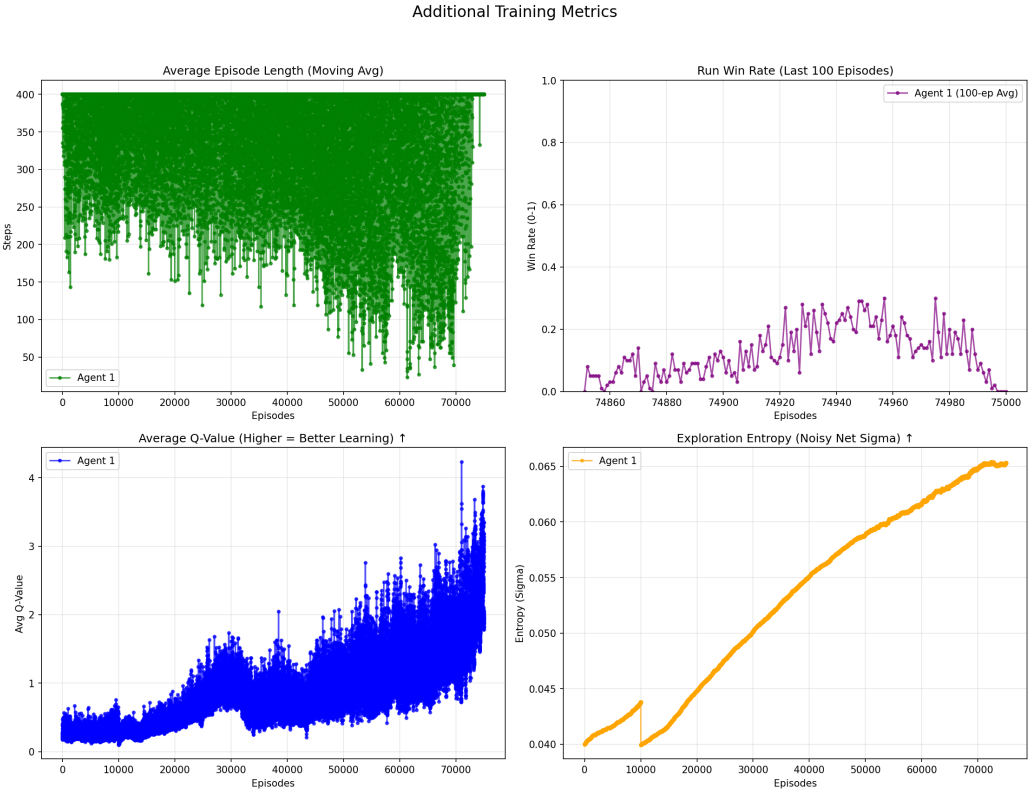


Figure B.18: Rainbow DQN+AAREN Extended Additional Metrics at 75,047 Episodes

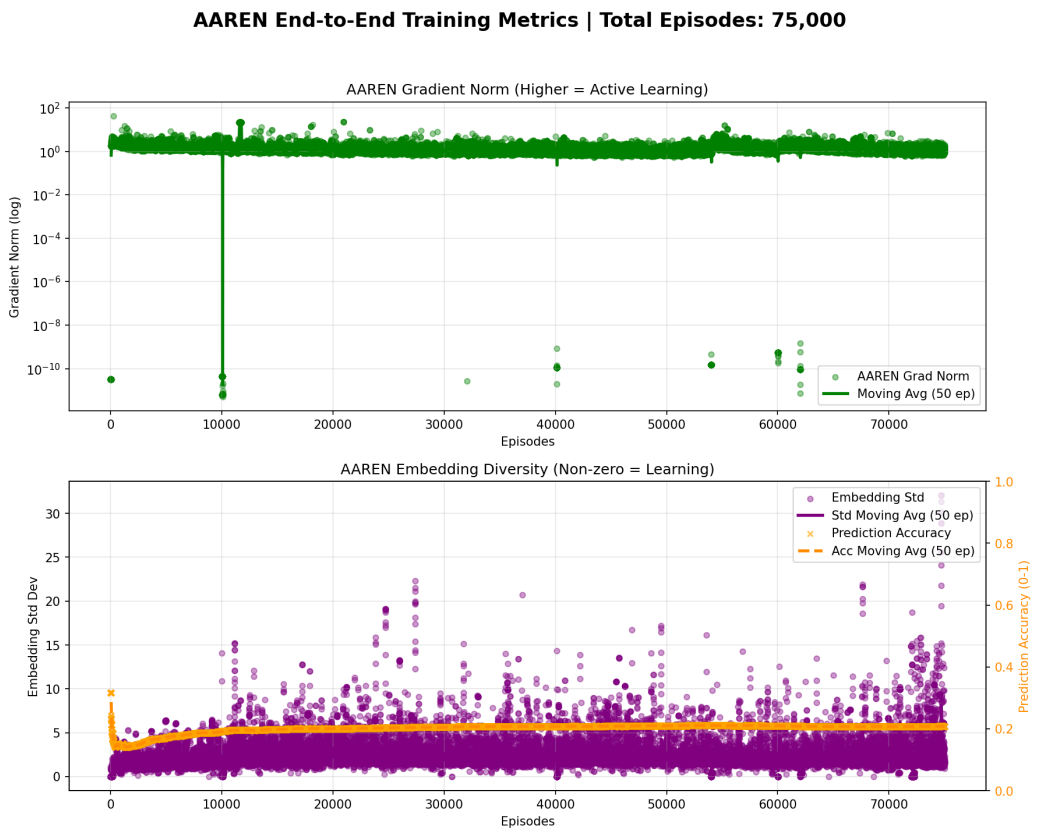


Figure B.19: AAREN End-to-End Training Metrics at 75,047 Episodes

B.3 AI Transparency, Inclusion, and Originality of Intent

The development of the MARQ framework for Stratego was governed by intentionality and human-led architectural design. While large language models and generative AI have become ubiquitous in modern software development, their role in this research was strictly limited to tasks such as grammatical refinement, high-level brainstorming, and code debugging for modules and networking. Every core component of the MARQ system, including the AAREN history aggregator and the ResNet-based policy heads, was researched, evaluated, and implemented with the intent of mastering imperfect information games in the Stratego domain.

REFERENCES

[1] Adams, G., Padmanabhan, S., and Shekhar, S. Resolving implicit coordination in multi-agent deep rl with deep q-networks & game theory. *arXiv preprint arXiv:2012.09136* (2023).

[2] Al-Selwi, S. M., Hassan, M. F., Abdulkadir, S. J., Muneer, A., Sumiea, E. H., Alqushaibi, A., and Ragab, M. G. Rnn-lstm: From applications to modeling techniques and beyond—systematic review. *Journal of King Saud University - Computer and Information Sciences 36*, 5 (2024), 102068.

[3] Amaliani, R. N., Soewardikoen, D. W., Azhar, H., and Rahman, Y. Designing board games as an enhancer of a sense of community and cognition for the elderly of tresna werdha budi pertiwi social care. *Advances in Social Humanities Research 3*, 4 (2025).

[4] Author, F., and Author, S. Bridging local and global knowledge via transformer in board games. In *Proceedings of the International Joint Conference on Artificial Intelligence* (2025).

[5] Barzilai, G., Shamir, O., and Zamani, M. Are convex optimization curves convex? *arXiv preprint arXiv:2503.10138* (2025).

[6] Becker, T., and Sunberg, Z. Imperfect information games and counterfactual regret minimization in space domain awareness. In *Advanced Maui Optical and Space Surveillance Technologies Conference (AMOS)* (2022), AMOS Technologies.

[7] Bellemare, M. G., Dabney, W., and Rowland, M. *Distributional Reinforcement Learning*. MIT Press, 2023.

[8] Blüml, J., Czech, J., and Kersting, K. Alphaze**: Alphazero-like baselines for imperfect information games are surprisingly strong. *Frontiers in Artificial Intelligence Volume 6 - 2023* (2023).

[9] Brim, A. Deep reinforcement learning pairs trading with a double deep q-network. In *Proceedings of the 10th Annual Computing and Communication Workshop and Conference (CCWC)* (Las Vegas, Nevada, USA, Jan.–2020 2020), pp. 222–227.

[10] Brown, N., and Sandholm, T. Solving imperfect-information games via subgame solving and depth-limited search. *Artificial Intelligence 297* (2021), 103503.

[11] Canese, L., Cardarilli, G. C., Di Nunzio, L., Fazzolari, R., Giardino, D., Re, M., and Spanò, S. Multi-agent reinforcement learning: A review of challenges and applications. *Applied Sciences 11*, 11 (2021), 4948. Article number; MDPI journal uses article numbers instead of page ranges.

[12] Chen, S., Fried, D., and Topcu, U. Human-agent cooperation in games under incomplete information through natural language communication, 2024.

[13] Chitayat, P. P., Colman, A. M., Hyde, R. J., Drachen, A., and Cowling, P. Wards: Modelling the worth of vision in MOBA's. In *Intelligent Systems and Applications - Proceedings of the 2020 Intelligent Systems Conference (IntelliSys)* (2020), vol. 1251 of *Advances in Intelligent Systems and Computing*, Springer, pp. 55–76.

[14] Dbouk, H. M., Ghorayeb, K., Kassem, H., Hayek, H., Torrens, R., and Wells, O. Facility placement layout optimization. *Journal of Petroleum Science and Engineering 207* (2021), 109079.

[15] Devlin, S., and Kudenko, D. Potential-based reward shaping in large-scale mdps. *Reinforcement Learning Journal 3*, 2 (2022), 145–167.

[16] Echchahed, A., and Castro, P. S. A survey of state representation learning for deep reinforcement learning. *Transactions on Machine Learning Research* (2025).

[17] El Shar, I., and Jiang, D. Weakly coupled deep q-networks. In *Advances in Neural Information Processing Systems 36* (2023), Curran Associates, Inc. NeurIPS 2023 Conference Track.

[18] Feng, L., Tung, F., Hajimirsadeghi, H., Ahmed, M. O., Bengio, Y., and Mori, G. Attention as an rnn. *arXiv preprint arXiv:2405.13956* (2024).

[19] Harrington, J. Let's meetup? board game communities in hong kong. *Games and Culture 20*, 3 (2025), 279–297.

[20] Hessel, M., Modayil, J., van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M., and Silver, D. Rainbow: Combining improvements in deep reinforcement learning. *Journal of Machine Learning Research 22*, 85 (2021), 1–52.

[21] Hu, C., Zhao, Y., Wang, Z., Du, H., and Liu, J. Games for artificial intelligence research: A review and perspectives. *IEEE Transactions on Artificial Intelligence 5*, 12 (2024), 5949–5968.

[22] Huang, Y. Deep q-networks. In *Deep Reinforcement Learning: Fundamentals, Research, and Applications*, S. Z. Hao Dong, Zihan Ding, Ed. Springer Nature, 2020, ch. 4, pp. 135–160. `http://www.deepreinforcementlearningbook.org`.

[23] Kaur, A., Kumar, K., Prakash, A., and Tripathi, R. Imperfect csi-based resource management in cognitive iot networks: A deep recurrent reinforcement learning framework. *IEEE Transactions on Cognitive Communications and Networking 9*, 5 (2023), 1271–1281.

[24] Kensert, A., Libin, P., Desmet, G., and Cabooter, D. Deep reinforcement learning for the direct optimization of gradient separations in liquid chromatography. *Journal of Chromatography A 1720* (2024), 464768.

[25] Le, N., Rathour, V. S., Yamazaki, K., Luu, K., and Savvides, M. Deep reinforcement learning in computer vision: A comprehensive survey, 2021.

[26] Li, B., Fang, Z., and Huang, L. Rl-cfr: Improving action abstraction for imperfect information extensive-form games with reinforcement learning. *arXiv preprint arXiv:2403.04344* (2024).

[27] Li, J., Zanuttini, B., and Ventos, V. Opponent-model search in games with incomplete information. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2024), vol. 38, pp. 9840–9847.

[28] Li, S., Zhang, Y., Wang, X., Xue, W., and An, B. Cfr-mix: Solving imperfect information extensive-form games with combinatorial action space. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI-21)* (2021), IJCAI, pp. 3663–3669.

[29] Li, Z., Ji, Q., Ling, X., and Liu, Q. A comprehensive review of multi-agent reinforcement learning in video games. *arXiv preprint arXiv:2509.03682v1* (2025).

[30] Luo, Z., Chen, Z., and Welsh, J. Multi-agent reinforcement learning with deep networks for diverse q-vectors. *arXiv preprint arXiv:2406.07848v1* (2024). Version v1, June 2024.

[31] Nair, R. R., Babu, T., Kishore, S., Pavithra, K., and Sumana, S. G. Improving alpha-beta pruning efficiency in chess engines. In *2024 International Conference on IT Innovation and Knowledge Discovery (ITIKD)* (Manama, Bahrain, 2025), IEEE, pp. 1–6.

[32] Novikov, A., and Yanovskyi, V. Analysis of the decision-making algorithm efficiency in complex game environments on the example of pac-man. *Information Technologies and Computer Engineering 3*, 21 (2024), 108–118.

[33] Ouyang, X., and Zhou, T. Imperfect-information game ai agent based on reinforcement learning using tree search and a deep neural network. *Electronics 12*, 11 (2023), 2453.

[34] Petersen, C. The reality of the board game community: And the connection, identity and experience that creates it. Master's thesis, Eastern University, Aug. 2024.

[35] Pérolat, J., et al. Mastering the game of stratego with model-free multiagent reinforcement learning. *Science 378*, 6623 (2022), 990–996.

[36] Qu, T., Zhou, Q., Zhu, J., and Duan, F. Strategy optimization of imperfect information games based on nfsp with ddqn. In *Advances in Guidance, Navigation and Control*, L. Yan, H. Duan, and Y. Deng, Eds., vol. 845 of *Lecture Notes in Electrical Engineering*. Springer, Singapore, 2023, pp. 4376–4383.

[37] Saffidine, A., Finnsson, H., and Buro, M. Alpha-beta pruning for games with simultaneous moves. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2021), vol. 26, pp. 556–562.

[38] Schmid, M., Moravčík, M., Burch, N., Kadlec, R., Davidson, J., Waugh, K., Bard, N., Timbers, F., Lanctot, M., Holland, G. Z., et al. Student of games: A unified learning algorithm for both perfect and imperfect information games. *Science Advances 9*, 46 (2023), eadg3256.

[39] Sokota, S., Vinitsky, E., Hu, H., Kolter, J. Z., and Farina, G. Superhuman ai for stratego using self-play reinforcement learning and test-time search, 2025.

[40] Sudhakar, A. V., Nekoei, H., Reymond, M., Liu, M., Rajendran, J., and Chandar, S. A generalist hanabi agent. *arXiv preprint arXiv:2503.14555v1* (2025).

[41] Wang, X., Li, Y., and Zhang, W. Deep reinforcement learning in imperfect-information games: A survey. *IEEE Transactions on Games 15*, 3 (2023), 421–438.

[42] Wang, Y. Game-theoretic understandings of multi-agent systems with multiple objectives, 2025.

[43] Wang, Y., Li, L., and Li, W. Constructing non-markovian decision process via history aggregator. *Applied Sciences 16*, 2 (2026), 955.

[44] Wang, Y., and Wellman, M. P. Regularization for strategy exploration in empirical game-theoretic analysis, 2023.

[45] Wongkamjan, W., Gu, F., Wang, Y., Hermjakob, U., May, J., Stewart, B., Kummerfeld, J., Peskoff, D., and Boyd-Graber, J. More victories, less cooperation: Assessing cicero's diplomacy play. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (2024), Association for Computational Linguistics, p. 12423–12441.

[46] Xie, S., and Li, Z. Implicit bias of adamw: ℓ_∞ norm constrained optimization. *arXiv preprint arXiv:2404.04454v1* (2024).

[47] Xu, H., Li, K., Fu, H., Fu, Q., and Xing, J. Autocfr: Learning to design counterfactual regret minimization algorithms. In *Proceedings of the Thirty-Sixth AAAI Conference on Artificial Intelligence (AAAI-22)* (2022), Association for the Advancement of Artificial Intelligence.

[48] Xu, H., Li, K., Fu, H., FU, Q., Xing, J., and Cheng, J. Dynamic discounted counterfactual regret minimization. In *The Twelfth International Conference on Learning Representations* (2024).

[49] Zakharenkov, A., and Makarov, I. Deep reinforcement learning with dqn vs. ppo in vizdoom. In *2021 IEEE 21st International Symposium on Computational Intelligence and Informatics (CINTI)* (Budapest, Hungary, 2021), IEEE, pp. 131–136.

[50] Zhang, B., and Sandholm, T. Subgame solving without common knowledge. *Advances in Neural Information Processing Systems 34* (2021), 23993–24004.

[51] Zhang, B. H., and Sandholm, T. General search techniques without common knowledge for imperfect-information games, and application to superhuman fog of war chess. *arXiv preprint arXiv:2506.01242* (2025).

[52] Zhang, K., Yang, Z., and Başar, T. Multi-agent reinforcement learning: A selective overview of theories and algorithms. *Handbook of reinforcement learning and control* (2021), 321–384.

[53] Zheng, S., Trott, A., Srinivasa, S., Naik, N., Gruesbeck, M., Parkes, D. C., and Socher, R. The ai economist: Improving equality and productivity with ai-driven tax policies. *arXiv preprint arXiv:2004.13332* (2020).

[54] Zhu, K., Liu, Q., Yan, X., Wu, F., Zhao, X., Zhou, P., and Yang, X. A comprehensive survey of multi-agent reinforcement learning. *IEEE Transactions on Neural Networks and Learning Systems 34*, 7 (2022), 3074–3093.

VITA

James Gabriel P. Mabagos is BS Computer Science student of the Department of Computer Science at the Ateneo de Naga University.

Mark Lawrence M. Quibot is BS Computer Science student of the Department of Computer Science at the Ateneo de Naga University.